

Data Structures

CSC 301

Lecture Notes

Computer Science
Osun State University
2025/2026

Prepared by
Tunde Sardine

Contents

Chapter 1: discuss the appropriate use of built-in data structures;	3
Chapter 2: apply object-oriented concepts (inheritance, polymorphism, design patterns, etc.) in	12
Chapter 3: implement various data structures and their algorithms, and apply them in implementing	20
Chapter 4: choose the appropriate data structure for modelling a given problem;	30
Chapter 5: analyse simple algorithms and determine their efficiency using big-O notation; and	39
Chapter 6: apply the knowledge of data structures to other application domains like data compression	50

Chapter 1

discuss the appropriate use of built-in data structures;

Computer programs manipulate data to solve real-world problems. The organization of data in computer memory directly dictates how efficiently algorithms can access, search, insert, and delete records. High-level programming languages provide **built-in data structures**, which are pre-packaged, highly optimized collection types supported natively by the language runtime and standard library.

Understanding the underlying mechanics of built-in data structures is vital for software performance and correctness. Selecting an inappropriate structure, such as using a dynamic array for frequent membership checks across millions of records, can transform a sub-second operation into an execution bottleneck lasting minutes. Knowing when to utilize each built-in structure enables engineers to write clean, performant code without reinventing common data management patterns.

This chapter examines the core built-in data structures found across modern high-level programming languages, including static arrays, dynamic arrays, tuples, dictionaries, and sets. It analyzes their physical memory layouts, operational time complexities, and behavioral trade-offs. Mastering these foundational structures provides the essential context required for object-oriented data structure design and formal algorithm analysis in subsequent chapters.

1.1 Memory Layout and Fundamental Characteristics

Built-in data structures organize data in physical system memory according to specific layout paradigms. These paradigms govern how memory is allocated, accessed, and modified during program execution.

1.1.1 Contiguous versus Non-Contiguous Allocation

Memory allocation strategies are broadly divided into contiguous and non-contiguous arrangements. A **contiguous memory allocation** reserves a single, uninterrupted block of physical memory addresses for storing data elements. Because elements sit directly

adjacent to one another, the memory address of the i -th element can be calculated directly from the base memory address of the structure using constant time address arithmetic:

$$\text{Address}(i) = \text{Base Address} + (i \times \text{Element Size}) \quad (1.1)$$

In contrast, non-contiguous memory allocation distributes data elements across arbitrary memory locations linked together via memory pointers. While non-contiguous structures facilitate dynamic resizing without reallocating full blocks, they lose the ability to perform constant-time random access by integer index and incur additional pointer overhead.

1.1.2 Mutability and Structural Invariance

Data structures are classified by whether their contents or dimensions can change after initialization. A **mutable** structure allows modification, addition, or removal of elements in place without allocating a new structure in memory. Dynamic arrays and dictionaries are primary examples of mutable built-in structures.

An **immutable** structure cannot be modified once created. Any transformation applied to an immutable structure creates a completely new instance in memory. Immutability provides inherent thread safety and guarantees structural invariance, making immutable built-in structures like tuples ideal for read-only configurations, dictionary keys, and concurrent execution contexts.

Static Array Layout (Fixed Memory Block = 4 Elements):

+-----+	+-----+	+-----+	+-----+
Address: 0x1000	Address: 0x1004	Address: 0x1008	Address: 0x100C
Index: 0	Index: 1	Index: 2	Index: 3
Value: 42	Value: 87	Value: 15	Value: 93
+-----+	+-----+	+-----+	+-----+

Dynamic Array Layout (Logical Size = 3, Allocated Capacity = 6):

+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Ind: 0	Ind: 1	Ind: 2	Unallocated Space	Unallocated Space	Unallocated Space
Val: A	Val: B	Val: C	(Reserved Buffer)	(Reserved Buffer)	(Reserved Buffer)
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
^ Base Pointer			^ Next Insertion Point		

1.2 Linear Built-in Data Structures

Linear data structures maintain a sequential ordering among their elements, where each element has a distinct predecessor and successor (except for the boundary elements).

1.2.1 Static Arrays

A static **array** is a fixed-size, contiguous sequence of elements of a single uniform data type. The total capacity of a static array must be known at allocation time and cannot grow or

shrink during program execution.

Because all elements share identical byte sizes, static arrays support direct constant-time indexed access, denoted as $O(1)$. However, inserting or deleting elements at arbitrary positions requires shifting subsequent elements, resulting in linear time complexity, $O(n)$, where n is the number of elements.

1.2.2 Dynamic Arrays

A **dynamic array** provides contiguous sequential storage similar to a static array but automatically resizes itself when capacity is exhausted. Popular high-level language implementations include Python `list`, Java `ArrayList`, and C++ `std::vector`.

To minimize the overhead of frequent allocations, dynamic arrays utilize a growth factor strategy. When an element is added to a full array, the underlying execution environment performs the following sequence:

1. Allocates a new contiguous memory block, typically twice the size of the existing block (growth factor of 2).
2. Copies all existing elements from the old memory block to the new memory block.
3. Releases the old memory block back to the system allocator.
4. Inserts the new element into the next available slot.

Dynamic Array Growth and Reallocation Sequence:

Step 1: Array is Full (Capacity = 4, Size = 4)

```
+---+---+---+---+
| A | B | C | D | (Old Memory Block)
+---+---+---+---+
```

Step 2: Append 'E' triggers allocation of new expanded block (Capacity = 8)

```
+---+---+---+---+---+---+---+---+
|   |   |   |   |   |   |   |   | (New Memory Block)
+---+---+---+---+---+---+---+---+
```

Step 3: Copy existing elements and insert 'E'

```
+---+---+---+---+---+---+---+---+
| A | B | C | D | E |   |   |   | (Size = 5, Capacity = 8)
+---+---+---+---+---+---+---+---+
                ^ Next Insertion Point
```

Step 4: Release old memory block back to heap allocator

Although individual resizing operations require $O(n)$ copy operations, amortized analysis proves that appending an element to a dynamic array executes in $O(1)$ time on average over a sequence of appends.

1.2.3 Tuples

A **tuple** is an ordered, immutable collection of elements. Unlike standard static arrays, tuples can store heterogeneous data types across their slots while maintaining fixed positional semantics.

Because tuples are immutable, the runtime allocates exact memory blocks with zero dynamic overhead or growth buffers. This makes tuples memory-efficient and faster to instantiate than dynamic arrays. Tuples are predominantly used to group related data fields together (such as database rows or multi-value function returns) without creating custom class abstractions.

1.3 Associative and Set Structures

Associative and set structures organize data based on logical keys or content values rather than sequential integer positioning.

1.3.1 Dictionaries and Hash Maps

A **dictionary** (also called a hash map or associative array) is an unordered collection of key-value pairs, where each key must be unique and immutable. Dictionaries allow programs to retrieve values rapidly using their associated keys rather than scanning through an entire dataset sequentially.

Internally, dictionaries rely on a mathematical function called a **hash function**. The hash function accepts a key as input and computes an integer hash code, which maps directly to a specific slot (bucket) within an underlying array.

Dictionary Hash-Based Lookup Mechanism:

```
Key: "OSUN_ID_402"
  |
  v
[ Hash Function ]  ==> Computed Bucket Index: 3
  |
  v
Underlying Bucket Array:
+---+-----+
| 0 | Empty |
+---+-----+
| 1 | Key: "OSUN_ID_101" -> Value: "A+" |
+---+-----+
| 2 | Empty |
+---+-----+
| 3 | Key: "OSUN_ID_402" -> Value: "B" | <== Match Found (0(1) Access)
+---+-----+
```

When two distinct keys produce the same bucket index, a hash collision occurs. Dictionaries resolve collisions using collision handling strategies such as **chaining** (maintaining a linked list per bucket) or **open addressing** (probing subsequent array slots). When a high-quality hash function is used, search, insertion, and deletion operations execute in $O(1)$ average time complexity.

1.3.2 Sets

A **set** is an unordered collection of unique elements. Sets mirror the mathematical properties of set theory, preventing duplicate values from existing within the same container.

Most language libraries implement sets using hash table backends (where elements serve as keys mapped to dummy values) or balanced search trees. Hash-based set implementations offer $O(1)$ average time complexity for insertion, deletion, and membership testing. Sets also provide direct, built-in operations for union ($\cup$), intersection ($\cap$), and difference ($\setminus$).

1.4 Comparative Performance Analysis and Selection Criteria

Choosing the correct built-in data structure requires balancing time complexity, space overhead, and programmatic constraints. Table 1.1 summarizes the asymptotic operational characteristics of fundamental built-in data structures.

To determine the optimal data structure for a given task, consider the following structural decision pipeline:

Structural Selection Decision Tree:

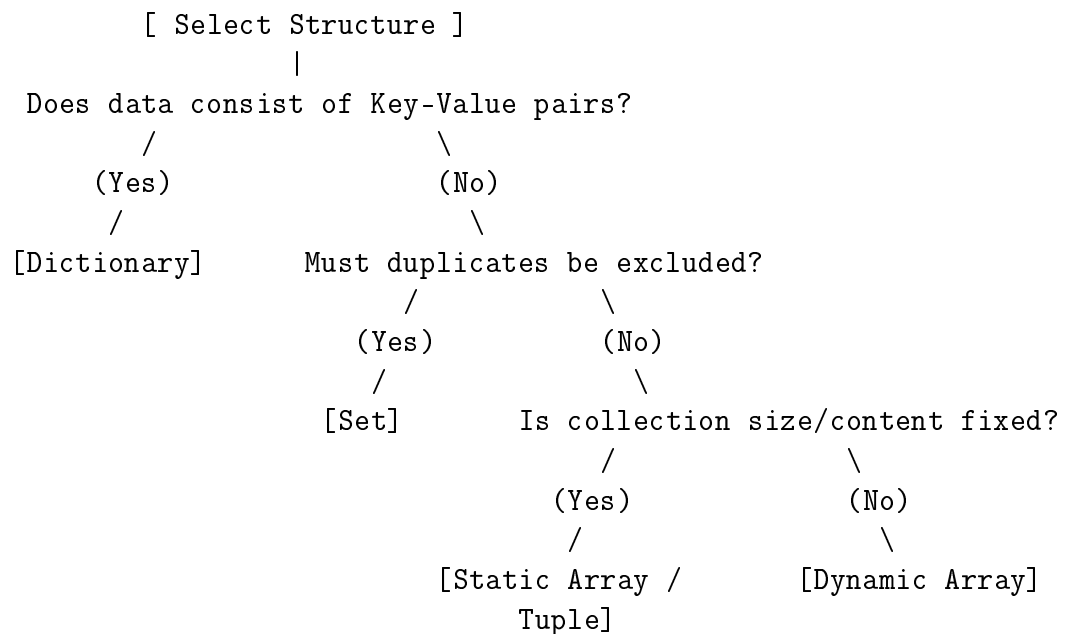


Table 1.1: Operational Complexity and Characteristics of Built-in Data Structures

Data Structure	Access by Index	Search by Value	Insertion / Deletion	Primary Use Case
Static Array	$O(1)$	$O(n)$	$O(n)$	Fixed-size numeric or fixed-length sequences.
Dynamic Array	$O(1)$	$O(n)$	$O(1)$ Amortized (at end)	General-purpose ordered sequences with variable size.
Tuple	$O(1)$	$O(n)$	Not Supported (Immutable)	Fixed multi-attribute records and composite lookup keys.
Dictionary	Not Supported	$O(1)$ Average	$O(1)$ Average	Rapid key-value lookups and relational mapping.
Set	Not Supported	$O(1)$ Average	$O(1)$ Average	Uniqueness enforcement and fast membership checks.

1. **Identity versus Association:** If data items are indexed by natural sequential integers, select a dynamic array or static array. If data items are retrieved via unique identifiers (strings, UUIDs, compound keys), select a dictionary.
2. **Uniqueness Requirements:** If duplicate elements must be filtered out automatically or membership tests occur frequently, select a set.
3. **Mutability Constraints:** If the collection must remain fixed after creation to prevent accidental modification, select a tuple.
4. **Access Pattern Optimization:** If elements are accessed via arbitrary integer positions, prioritize linear contiguous structures to leverage hardware CPU cache lines.

1.5 Worked Examples

The following concrete engineering scenarios illustrate how to evaluate and select built-in data structures based on performance requirements.

1.5.1 Worked Example 1: Efficient Student Record Management

Problem Statement: A university registrar system receives continuous batches of student records. The system must perform two core tasks efficiently:

1. Retrieve student details instantly given a unique 8-digit Student Identification Number (Matric Number).
2. Maintain a log of incoming records in the exact order they arrive for auditing purposes.

Analysis and Solution: Using a dynamic array for Task 1 requires scanning elements sequentially, yielding an $O(n)$ search time. For 100,000 students, scanning the array repeatedly creates severe performance penalties.

To achieve optimal performance, the system should combine two complementary built-in structures:

- **Dictionary for Fast Lookups:** Map each unique Matric Number (string key) to the corresponding student record object (value). This provides $O(1)$ average time retrieval for Task 1.
- **Dynamic Array for Audit Logging:** Append incoming student record references sequentially to a dynamic array. This preserves insertion order and completes Task 2 in $O(1)$ amortized time per insertion.

1.5.2 Worked Example 2: Filtering Duplicate IP Addresses

Problem Statement: A web server security daemon processes a log stream containing 10,000,000 IP address records. The log contains many repeating IP addresses. The daemon must compute the exact number of unique IP addresses present in the log and check whether specific suspicious IP addresses are present.

Analysis and Solution: If a dynamic array is used, checking if an IP address is already recorded requires searching through existing elements, taking $O(k)$ time for an array of size k . Building the unique list across n log entries takes $O(n^2)$ total operations:

$$\text{Total Operations} = \sum_{k=1}^n k = \frac{n(n+1)}{2} \approx 5 \times 10^{13} \text{ operations}$$

By choosing a **Set** structure instead:

1. Each IP address string is inserted into the set. The set hashes the string and performs insertion in $O(1)$ average time. Duplicates are discarded automatically.
2. Total time complexity to process n entries drops to $O(n)$, executing roughly 10^7 operations instead of 5×10^{13} .
3. Subsequent queries checking whether a suspicious IP exists execute in $O(1)$ average time.

1.6 Summary and Next Steps

Built-in data structures form the essential execution foundation of software applications. Arrays and dynamic arrays provide fast index-based positional access; tuples enforce immutability and record integrity; dictionaries enable constant-time key-based lookup; and sets optimize uniqueness checking and set-theoretic operations. Choosing the appropriate structure based on temporal and spatial complexity prevents severe performance bottlenecks.

While built-in data structures solve standard data management problems, complex domains often demand domain-specific data structures with customized behavior. The next chapter introduces object-oriented programming concepts, demonstrating how encapsulation, inheritance, and polymorphism enable engineers to build robust, custom data abstractions.

Tutorial Questions

1. Explain the fundamental memory organization difference between contiguous and non-contiguous data structures. Identify how this difference impacts index-based element access time.
2. A software developer uses a dynamic array to store historical transaction records. Describe the internal steps taken by the runtime environment when an item is added to a dynamic array that has reached its maximum allocated capacity. What is the amortized time complexity of this operation?
3. Compare and contrast a tuple and a dynamic array across three distinct technical dimensions: memory allocation efficiency, mutability, and primary software usage cases.

4. An application performs frequent membership queries ("does item X exist in collection C ?") on a dataset containing 2,000,000 records. Evaluate the operational efficiency of performing this query using a dynamic array versus a set. Provide big-O notation complexities for both approaches.
5. Explain how a dictionary achieves $O(1)$ average time complexity for value retrieval. What condition causes this performance to degrade to $O(n)$, and how do modern hashing implementations mitigate this risk?
6. A system tracks live vehicle positions using geographical coordinate pairs (latitude, longitude). These coordinates must be updated continuously, but specific route history checkpoints must remain permanent and unalterable. Recommend the built-in data structure best suited for storing live coordinates versus permanent route checkpoints, justifying your selection for each.
7. Analyze the computational complexity trade-offs involved when using a static array versus a dynamic array in a memory-constrained embedded system with known, fixed bounds.

Chapter 2

apply object-oriented concepts (inheritance, polymorphism, design patterns, etc.) in

Data structures serve as the structural backbone of software systems, defining how memory is organized and manipulated. However, raw data structures written without proper abstraction mechanisms can become tightly coupled to specific programs, making code difficult to maintain, extend, or reuse. Object-oriented programming provides architectural principles that decouple abstract behavioral contracts from concrete physical layouts.

This chapter examines how key object-oriented concepts, specifically inheritance, polymorphism, and design patterns, are applied to data structure design. By leveraging these concepts, developers can construct modular data structure libraries that decouple client interfaces from underlying operational mechanics.

Understanding these abstraction techniques is essential for modern software development. Building upon the built-in data structures explored in Chapter 1, this material establishes the structural patterns used to build custom, extensible data structures in Chapter 3 and beyond.

2.1 Object-Oriented Abstraction and Abstract Data Types

In software development, an **abstract data type** (ADT) is a mathematical specification of a collection of data and the operations that can be performed on that data. An ADT defines *what* operations are available, their parameters, and their expected outcomes, without specifying *how* those operations are implemented in memory. A concrete data structure, by contrast, is the physical implementation of an ADT in code, specifying data fields, memory allocation strategies, and algorithmic routines.

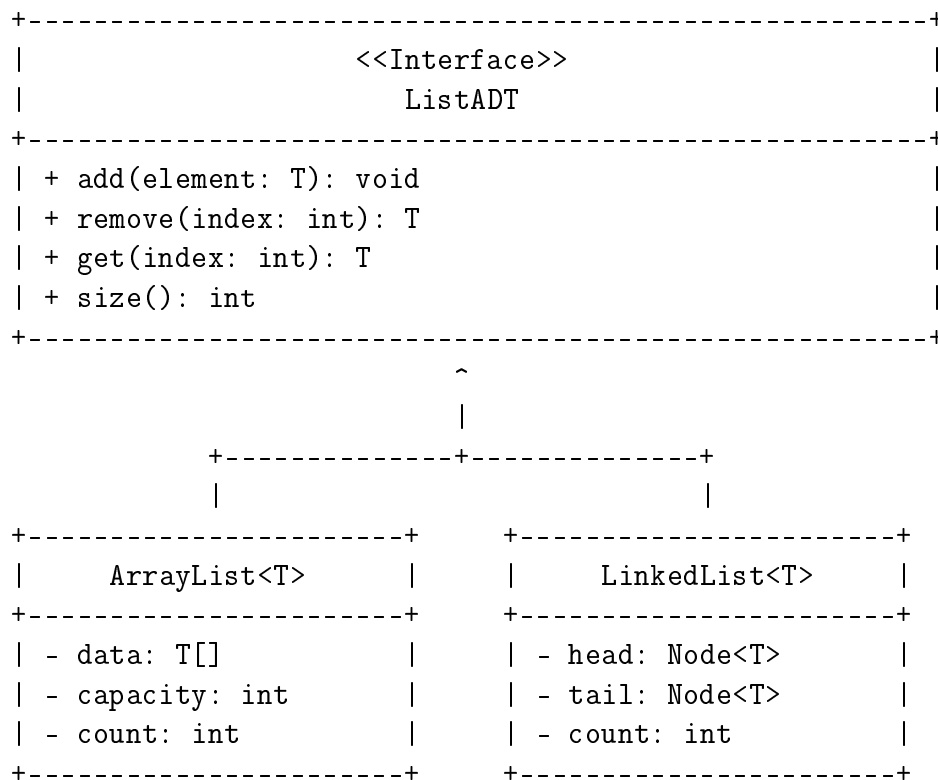
Object-oriented programming enforces ADT specifications through encapsulation and access control. Encapsulation isolates the internal memory representation of a data structure behind a public interface. Private fields prevent external code from directly modifying pointer references or array indices, preserving internal structural invariants.

2.1.1 Separating Specification from Implementation

Consider a sequence of elements accessible by index. The ADT specification defines operations such as insertion, removal, search, and size retrieval. Multiple distinct concrete data structures can satisfy this same specification:

- A **dynamic array**, which stores elements in contiguous memory and offers fast positional access.
- A linked list, which stores elements in disjoint nodes connected by pointers, offering fast insertions at node boundaries.

By defining an abstract base class or interface, client programs can operate directly on the abstraction. This allows software systems to substitute concrete representations without altering the client code that depends on them.



2.2 Inheritance and Polymorphism in Data Structure Design

Inheritance is an object-oriented mechanism where a child class acquires fields and methods from a parent class. In data structure engineering, inheritance serves two primary roles: code reuse and subtype specification.

Subtyping inheritance defines formal class hierarchies where child classes fulfill the contract of a parent type. Implementation inheritance allows derived classes to inherit common operational logic, reducing duplicate code across related structures.

Polymorphism enables objects of different concrete classes to respond to the same method call in class-specific ways. Through dynamic method dispatch, the runtime environment inspects the concrete type of an object and invokes the corresponding method implementation.

2.2.1 Class Hierarchies and Code Sharing

Inheritance hierarchies prevent redundancy by placing shared operational logic high in the class tree. An abstract base class can implement operations that depend only on other abstract primitives.

For example, an abstract list base class can provide a concrete implementation of an `isEmpty()` method that checks if `size() == 0`. Subclasses inheriting this base class receive `isEmpty()` automatically, needing only to implement the storage-specific `size()` method.

Table 2.1: Comparison of Inheritance and Composition in Data Structure Design

Dimension	Inheritance (“is-a”)	Composition (“has-a”)
Relationship	Establishes a rigid subtype hierarchy between parent and child classes.	Embeds an instance of another data structure as an internal field.
Coupling	High coupling; modifications to base classes affect all derived classes.	Low coupling; the encapsulated instance can be swapped independently.
Flexibility	Static; behavior is determined at compile time by class definitions.	Dynamic; contained objects and strategy delegates can be updated at runtime.
Primary Purpose	Defining shared interface contracts and inheriting shared implementations.	Building complex or restricted structures from simple components.

While inheritance creates tight coupling between base and child classes, composition embeds a data structure inside another class, delegating work to the internal object. Designing data structures often requires choosing between inheritance and composition to maintain flexibility.

2.3 Applying Design Patterns to Data Structures

Design patterns are reusable, generalized software engineering solutions to recurring design problems within a given domain. Applying design patterns to data structures yields modular, flexible implementations that adapt easily to changing requirements.

Four essential design patterns used extensively in data structure design are:

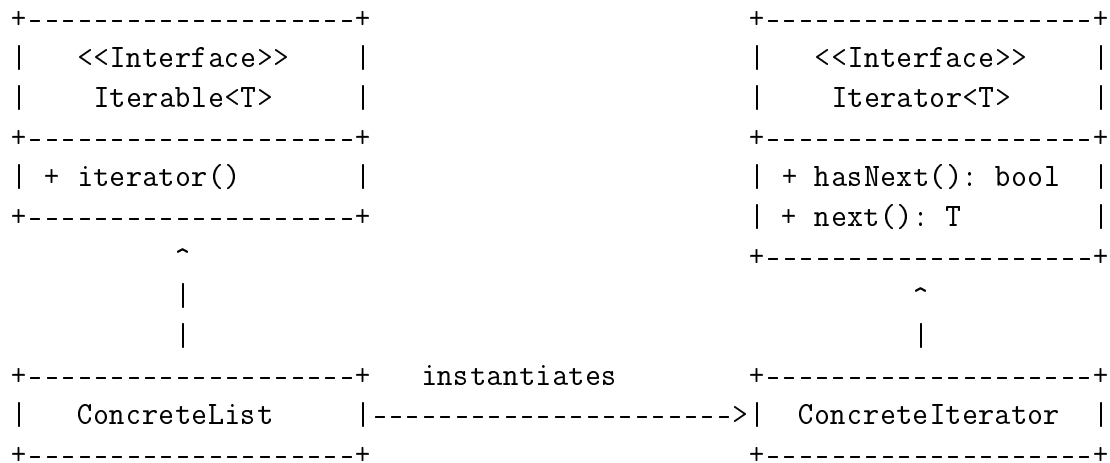
1. **Iterator Pattern:** Traverses aggregate elements sequentially without exposing internal structural representations.

2. **Factory Pattern:** Encapsulates object creation mechanics for nodes and structural components.
3. **Strategy Pattern:** Parameterizes key algorithmic behaviors within a data structure.
4. **Adapter Pattern:** Converts an existing data structure interface into an interface expected by client code.

2.3.1 The Iterator Pattern

The **iterator pattern** separates element traversal from the internal layout of a collection. Without an iterator, client code traversing a linked list must manually navigate pointers, while traversing an array requires array index management.

An iterator provides a unified, sequential interface featuring operations such as `hasNext()` and `next()`. Client code can iterate over arrays, trees, or graphs using identical loop structures.



The iterator preserves data structure integrity by preventing client code from modifying internal pointers directly during traversal.

2.3.2 The Factory Pattern

The **factory pattern** delegates object instantiation to dedicated methods or builder classes. In complex data structures, creating nodes or structural elements directly inside algorithms couples the structure to concrete node classes.

For example, a balanced search tree class can use a factory method, `createNode(key, value)`, to generate tree nodes. Subclasses override this factory method to instantiate specialized nodes, such as Red-Black nodes containing color bits, without altering the core search tree logic.

2.3.3 The Strategy Pattern

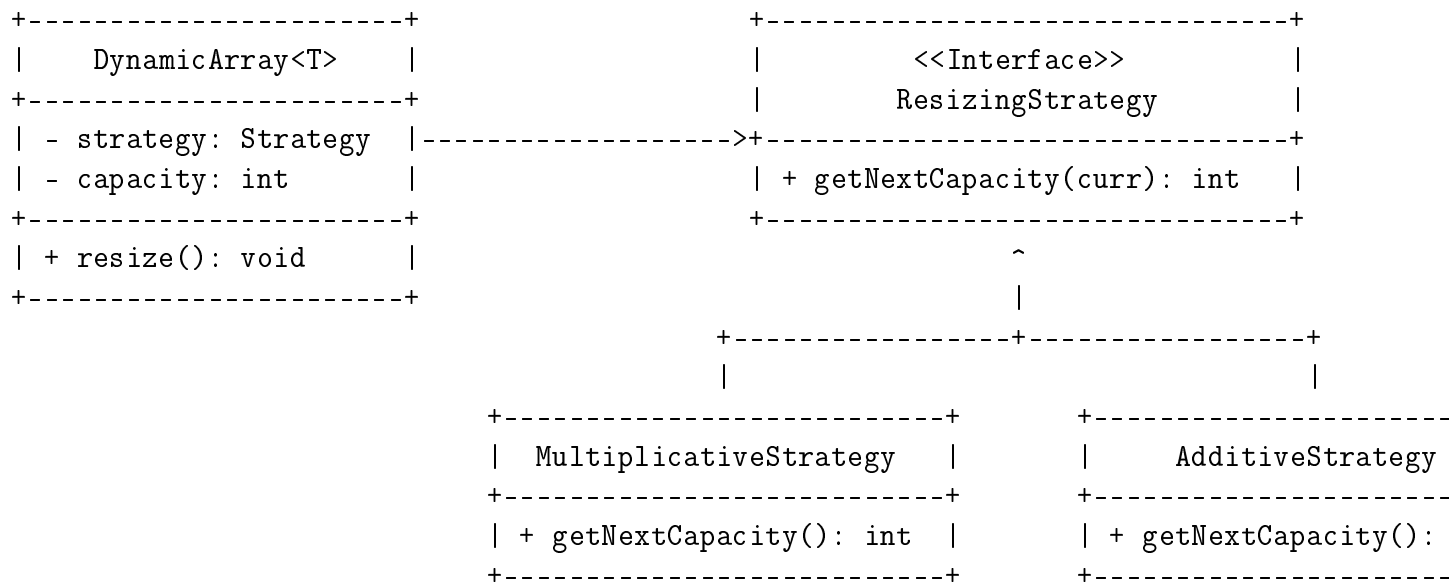
The **strategy pattern** encapsulates interchangeable algorithms into separate classes conforming to a shared interface. Data structures often rely on external algorithms to drive

internal behavior, including:

- Custom comparison operations for sorting and heap ordering.
- Alternative hash functions for hash tables.
- Dynamic resizing strategies for dynamic arrays.

By passing a strategy object to a data structure during construction, the structure's runtime behavior can be changed without modifying its source code.

For instance, a dynamic array can accept a growth strategy interface. One strategy can grow capacity multiplicatively by doubling ($1.5\times$ or $2.0\times$), while another increases capacity linearly by fixed increments.



2.3.4 The Adapter Pattern

The **adapter pattern** converts the interface of an existing class into a different interface expected by a client. This pattern is commonly used to implement restricted access data structures, such as Stacks and Queues, using existing linear structures.

Consider implementing a Stack ADT using a dynamic array. A Stack requires `push()`, `pop()`, and `peek()` operations, whereas a dynamic array exposes indexed access operations like `addAt()`, `removeAt()`, and `get()`. The Stack adapter class wraps an internal dynamic array instance, translating stack operations into array actions.

```

class StackAdapter<T>:
    private list: DynamicArray<T>

    constructor():
        list = new DynamicArray<T>()

    method push(item: T): void:

```



```

list.addLast(item)

method pop(): T:
    if list.isEmpty():
        throw UnderflowException
    return list.removeLast()

method peek(): T:
    if list.isEmpty():
        throw UnderflowException
    return list.get(list.size() - 1)

```

2.4 Performance Trade-Offs and Architectural Considerations

Applying object-oriented abstractions to data structures introduces performance trade-offs that must be evaluated against engineering advantages.

Table 2.2: Summary of Core Design Patterns in Data Structure Design

Pattern	Category	Structural Context	Primary Benefit
Iterator	Behavioral	Collection element traversal.	Decouples collection traversal from internal layout.
Factory	Creational	Node and container creation.	Encapsulates object creation and enables specialization.
Strategy	Behavioral	Hashing, growth, and sorting routines.	Enables runtime selection of dynamic algorithms.
Adapter	Structural	Stack, Queue, and Deque construction.	Reuses existing container code via interface transformation.

2.4.1 Virtual Method Overhead and Memory Layout

Dynamic polymorphism requires runtime method dispatch. In languages that compile to virtual machine bytecode or native binaries, polymorphic calls use virtual method tables (vtables) to locate method pointers at runtime. This indirect call mechanism introduces minor latency compared to direct function calls and can prevent CPU compiler optimizations such as inline expansion.

Additionally, object-oriented abstractions introduce memory overhead:

- Object headers: Language runtimes attach metadata, such as type indicators and synchronization locks, to every object instance.

- Indirection and cache performance: Storing collections of objects via pointer references can cause CPU cache misses due to disjoint memory locations, compared to contiguous arrays of primitive data types.

Contiguous Primitive Array Layout (Cache-Friendly):

```
+-----+-----+-----+-----+
| Data 1 | Data 2 | Data 3 | Data 4 |
+-----+-----+-----+-----+
```

Reference-Based Object Layout (Pointer Indirection):

```
+-----+-----+-----+-----+
| Ref 1  | Ref 2  | Ref 3  | Ref 4  |
+-----+-----+-----+-----+
|        |        |        |        |
|   v    |   v    |   v    |   v    |
+-----+-----+-----+-----+
|Obj 1|  |Obj 2|  |Obj 3|  |Obj 4|
+-----+-----+-----+-----+
```

Engineers must weigh clean architectural design against low-level execution speed. For general application development, object-oriented abstractions improve code maintainability, safety, and reusability. In performance-critical systems, direct memory layouts and simplified procedural abstractions may be preferred.

2.5 Summary

Object-oriented programming concepts provide tools for engineering reusable data structure libraries. Abstract data types separate interface definitions from concrete storage mechanics. Inheritance establishes formal type hierarchies and shares common implementations, while polymorphism allows applications to manipulate collections generically. Design patterns like Iterator, Factory, Strategy, and Adapter solve common architectural problems, improving data structure flexibility and reuse.

These principles pave the way for Chapter 3, where we will build fundamental data structures, including stacks, queues, linked lists, trees, and graphs, from the ground up using object-oriented principles.

Tutorial Questions

1. Explain the distinction between an Abstract Data Type (ADT) and a concrete data structure. Provide two distinct concrete data structure examples for a single ADT specification.
2. Describe how encapsulation helps protect the internal structural invariants of a complex data structure. What vulnerabilities arise when internal representation details are exposed?

3. Compare inheritance (“is-a”) and composition (“has-a”) as techniques for constructing new data structures. In what scenario is composition preferred over inheritance?
4. Illustrate the Iterator Pattern using a class diagram or pseudocode. How does the iterator pattern decouple client code from the physical storage representation of a collection?
5. An engineer needs to implement a bounded Queue structure using an existing `DynamicArray` class. Identify which design pattern should be applied, write pseudocode for the class wrapper, and explain how the wrapper enforces queue behavior.
6. Discuss the Strategy Pattern in the context of dynamic array resizing. Design a scenario where switching dynamic array growth strategies at runtime improves performance or memory efficiency.
7. Analyze the performance costs associated with applying polymorphism and abstract interface abstractions to high-performance data structures. Identify two computational or memory overheads introduced by object-oriented abstraction layers.

Chapter 3

implement various data structures and their algorithms, and apply them in implementing

Building upon the abstract data types and object-oriented design patterns introduced in prior chapters, software systems require concrete underlying implementations to manage memory and execute operations efficiently. While high-level programming environments provide built-in data structures, constructing these structures directly exposes fundamental trade-offs between memory layout, access overhead, and algorithmic complexity.

This chapter examines the internal mechanics, structural implementations, and algorithmic operations of core data structures: linked lists, stacks, queues, binary search trees, and hash tables. Mastering these low-level implementations provides the operational insight needed to evaluate execution bottlenecks, avoid hidden overheads, and build custom data organizations for complex software applications.

3.1 Pointer-Based Linear Structures: Linked Lists

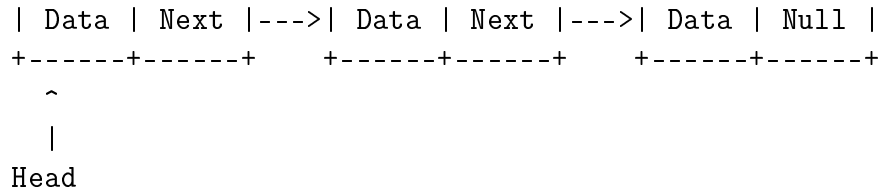
Linear data structures organize elements in a sequential order. Arrays allocate contiguous blocks of memory, whereas pointer-based lists distribute elements dynamically across non-contiguous physical memory locations.

3.1.1 Singly Linked Lists

A **singly linked list** consists of a sequence of nodes where each node contains two distinct fields: a data element and a reference pointer to the next node in the sequence. The structure is anchored by a head pointer that points to the first node. The terminal node contains a null reference to signal the end of the list.

Singly Linked List Memory Layout:

+-----+-----+ +-----+-----+ +-----+-----+



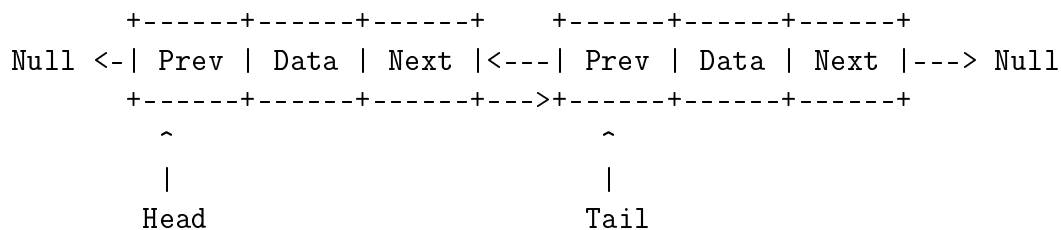
Primary operations on a singly linked list operate via pointer manipulation:

- **Insertion at Head:** A new node is allocated, its next pointer is directed to the current head, and the head pointer is updated to point to the new node. This operates in $O(1)$ constant time.
- **Traversal and Search:** Accessing an element at index k requires traversing k pointer references sequentially starting from head. This operation requires $O(n)$ linear time.
- **Deletion:** Removing a target node requires updating the next pointer of its preceding node to bypass the target node and point directly to the target node's successor. Locating the predecessor requires $O(n)$ search time, but the link modification itself is $O(1)$.

3.1.2 Doubly Linked Lists

A **doubly linked list** expands the node design by incorporating two pointers per node: a next pointer referencing the successor node and a prev pointer referencing the predecessor node. A tail pointer is maintained alongside the head pointer to anchor both ends of the structure.

Doubly Linked List Memory Layout:



Bidirectional pointers eliminate the need to traverse from the head when searching for a preceding node during deletion. Given a direct pointer reference to a node P within a doubly linked list, P can be removed in $O(1)$ time by setting $P.\text{prev.next} = P.\text{next}$ and $P.\text{next.prev} = P.\text{prev}$. However, this additional pointer maintenance increases the per-node dynamic memory overhead.

3.1.3 Worked Example: Deletion in a Doubly Linked List

Consider a doubly linked list containing nodes $A \leftrightarrow B \leftrightarrow C$. To delete target node B :

1. Access node A via $B.\text{prev}$ and node C via $B.\text{next}$.
2. Update node A 's forward reference: $A.\text{next} \leftarrow C$.

3. Update node C 's backward reference: $C.\text{prev} \leftarrow A$.
4. Clear node B 's internal pointers: $B.\text{next} \leftarrow \text{null}$, $B.\text{prev} \leftarrow \text{null}$.
5. Release memory allocated for node B .

3.2 Restricted Linear Structures: Stacks and Queues

Stacks and queues apply specific constraints to element access and ordering, abstracting insertion and removal into well-defined interface behaviors.

3.2.1 Stacks

A **stack** is a linear data structure operating under a Last-In, First-Out (LIFO) protocol. Elements can only be added or removed from a single end, designated as the top of the stack.

The core operations are:

- `push(item)`: Places a new item onto the top of the stack.
- `pop()`: Removes and returns the top item from the stack.
- `peek()`: Returns the top item without modifying the stack.

A stack can be implemented using either a dynamic array or a singly linked list. In an array-based implementation, the top is tracked by an integer index, yielding $O(1)$ amortized push operations and $O(1)$ pop operations. In a linked list implementation, push and pop operations perform node insertion and removal at the list head in deterministic $O(1)$ time.

Stack Conceptual Structure:

```

+-----+
| Item C | <-- Top of Stack (Push / Pop)
+-----+
| Item B |
+-----+
| Item A |
+-----+

```

3.2.2 Queues

A **queue** is a linear data structure operating under a First-In, First-Out (FIFO) protocol. Elements are inserted at the rear and removed from the front.

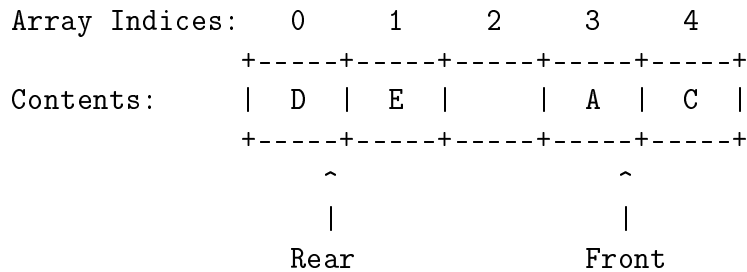
The core operations are:

- `enqueue(item)`: Appends an element to the rear of the queue.
- `dequeue()`: Removes and returns the element at the front of the queue.

Implementing a queue via a standard contiguous array can cause performance issues if dequeue operations shift remaining elements left, incurring $O(n)$ cost per operation. To maintain $O(1)$ operations, arrays are implemented as **circular queues**, where two index pointers (front and rear) wrap around the physical boundaries using modular arithmetic:

$$\text{rear} = (\text{rear} + 1) \pmod{\text{capacity}}$$

Circular Queue Array Implementation:



3.2.3 Worked Example: Postfix Expression Evaluation using a Stack

Postfix notation (Reverse Polish Notation) eliminates the need for parentheses in arithmetic expressions by placing operators immediately after their operands. Evaluators process postfix strings sequentially from left to right using a stack:

- If a value is an operand, push it onto the stack.
- If a value is an operator, pop two operands from the stack, apply the operator (the second popped value is the left operand), and push the result back onto the stack.

Evaluate the postfix expression: 6 2 3 * + 4 -

Table 3.1: Step-by-step Evaluation of 6 2 3 * + 4 -

Step	Token	Action	Stack State (Bottom to Top)
1	6	Push 6	[6]
2	2	Push 2	[6,2]
3	3	Push 3	[6,2,3]
4	*	Pop 3,2; Evaluate 2 * 3 = 6; Push 6	[6,6]
5	+	Pop 6,6; Evaluate 6 + 6 = 12; Push 12	[12]
6	4	Push 4	[12,4]
7	-	Pop 4,12; Evaluate 12 - 4 = 8; Push 8	[8]

The evaluation completes with a single final value 8 remaining on top of the stack.

3.3 Nonlinear Structures: Binary Search Trees

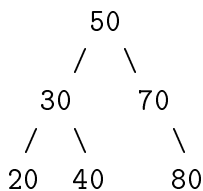
Linear data structures require linear search time $O(n)$ for unsorted data. Hierarchical structures optimize search, insertion, and deletion times through branched structural paths.

3.3.1 Binary Search Tree Property

A **binary search tree** (BST) is a hierarchical, node-based tree structure. Each node contains a key, a satellite payload, a left reference pointer, and a right reference pointer. A binary tree satisfies the binary search tree property if for every node N :

- Every node key in the left subtree of N is strictly less than N 's key.
- Every node key in the right subtree of N is strictly greater than N 's key.

Binary Search Tree Layout:



3.3.2 Core Operations and Algorithms

Search and Insertion

Searching begins at the root node. The search key is compared against the current node key:

- If equal, the node is returned.
- If the search key is smaller, search recursively continues into the left child subtree.
- If the search key is larger, search recursively continues into the right child subtree.

Insertion follows the same structural navigation path until reaching a null link, where the new node is attached as a leaf node.

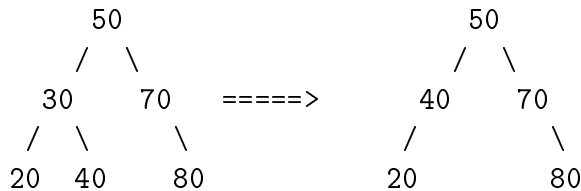
Deletion

Tree node deletion must preserve the BST ordering invariant across three possible node configuration cases:

1. **Node has no children (Leaf Node):** Remove the node directly by setting its parent reference to null.
2. **Node has one child:** Replace the target node with its single child by pointing the target node's parent directly to that child.

3. **Node has two children:** Locate the node's **in-order successor** (the node with the smallest key in its right subtree). Copy the in-order successor's key and satellite payload into the target node, then recursively delete the in-order successor node from its original position (which is guaranteed to fall under Case 1 or Case 2).

Deletion of Node with Two Children (Deleting Node 30):

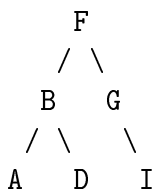


3.3.3 Tree Traversals

Tree traversal algorithms visit every node in a tree systematically in $O(n)$ time:

- **In-Order Traversal (Left, Node, Right):** Recursively visits the left subtree, processes the current node, and visits the right subtree. On a BST, this outputs keys in ascending sorted order.
- **Pre-Order Traversal (Node, Left, Right):** Processes the current node before visiting its subtrees. Used to clone or serialize tree topologies.
- **Post-Order Traversal (Left, Right, Node):** Processes both subtrees prior to evaluating the current node. Used in bottom-up memory deallocation and mathematical syntax tree evaluation.

Tree Traversal Example:



In-Order (Left, Node, Right): A, B, D, F, G, I

Pre-Order (Node, Left, Right): F, B, A, D, G, I

Post-Order (Left, Right, Node): A, D, B, I, G, F

3.4 Associative Mapping: Hash Tables

A **hash table** is an associative data structure designed to provide constant average-time $O(1)$ key search, insertion, and deletion. It maps keys to array index locations using a mathematical transformation called a hash function.

3.4.1 Hash Functions and Collisions

A hash function $h(k)$ maps an arbitrary key space k to a bounded integer range $[0, M - 1]$, where M is the size of the underlying array table. A uniform hash function distributes keys evenly across all array indices to minimize collisions.

A **hash collision** occurs when two distinct keys evaluate to identical indices ($h(k_1) = h(k_2)$ for $k_1 \neq k_2$). Because key spaces are typically much larger than array size M , collisions are unavoidable due to the Pigeonhole Principle.

3.4.2 Collision Resolution Techniques

Separate Chaining

In separate chaining, each array slot acts as a reference to an independent linked list (or bucket) storing all key-value entries mapped to that index.

Separate Chaining Layout:

```
Slot
+---+
| 0 | --> [ Null ]
+---+
| 1 | --> [ KeyA | ValA ] ---> [ KeyB | ValB ] ---> Null
+---+
| 2 | --> [ Null ]
+---+
```

Search time in a chained table depends on the average list length, governed by the **load factor** $\alpha = n/M$, where n is the total number of inserted keys and M is table capacity.

Open Addressing

Open addressing stores all entries directly inside the primary array. When a collision occurs, the algorithm systematically probes alternate slots until an empty cell is found:

- **Linear Probing:** Inspects sequential slots using index formula $h(k, i) = (h'(k) + i) \pmod{M}$ for probe step $i \in \{0, 1, \dots, M - 1\}$. This approach can lead to primary clustering, where contiguous blocks of occupied slots build up and slow down lookups.
- **Quadratic Probing:** Uses a non-linear quadratic offset $h(k, i) = (h'(k) + c_1i + c_2i^2) \pmod{M}$ to reduce primary clustering.
- **Double Hashing:** Uses a secondary independent hash function $h_2(k)$ to calculate probe step sizes: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod{M}$.

3.4.3 Worked Example: Collision Resolution in Hash Tables

Insert integer keys [15, 22, 8, 29] into a hash table of size $M = 7$ using hash function $h(k) = k \pmod{7}$.

- Insert 15: $h(15) = 15 \pmod{7} = 1$. Slot 1 is empty. Place 15 at index 1.
- Insert 22: $h(22) = 22 \pmod{7} = 1$. Collision at slot 1.
- Insert 8: $h(8) = 8 \pmod{7} = 1$. Collision at slot 1.
- Insert 29: $h(29) = 29 \pmod{7} = 1$. Collision at slot 1.

Using **Linear Probing**:

1. Key 15 goes to slot 1.
2. Key 22 hashes to 1 (occupied). Probe index $(1 + 1) \pmod{7} = 2$. Place 22 at slot 2.
3. Key 8 hashes to 1 (occupied). Probe index 2 (occupied). Probe index $(1 + 2) \pmod{7} = 3$. Place 8 at slot 3.
4. Key 29 hashes to 1 (occupied). Probe index 2 (occupied), index 3 (occupied), index $(1 + 3) \pmod{7} = 4$. Place 29 at slot 4.

Linear Probing Result Table:

Index:	0	1	2	3	4	5	6
Keys:	Null	15	22	8	29	Null	Null

Using **Separate Chaining**:

Separate Chaining Result Table:

Index 1: [15] ---> [22] ---> [8] ---> [29] ---> Null

3.5 Comparative Analysis and Systemic Applications

Selecting the right data structure requires evaluating operational performance, memory overhead, and implementation trade-offs.

*Note: Insertion and deletion operations for linked lists run in $O(1)$ time given a direct pointer reference to the target position; otherwise, locating the position requires $O(n)$ search time.

3.5.1 Integrated Software Applications

Real-world applications combine these core structures to manage operational tasks efficiently:

- **Text Editor Undo/Redo System:** Dual stack structures store operational state histories. Undoing an action pops state data from the undo stack and pushes it onto the redo stack.

Table 3.2: Algorithmic Time and Space Complexity Comparison

Data Structure	Average Access	Average Search	Average Insertion	Average Deletion
Singly Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^*$	$\Theta(1)^*$
Doubly Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^*$	$\Theta(1)^*$
Stack (Array/List)	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Queue (Array/List)	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Binary Search Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Table	N/A	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

- **Task Scheduling and Breadth-First Search (BFS):** Operating system process managers and graph traversal algorithms use queues to maintain execution order and process nodes sequentially.
- **Database Indexing and Symbol Tables:** Compilers use hash tables for constant-time lookup in scope symbol tables, while binary search trees support range queries and sorted index scans.

3.6 Conclusion

Concrete data structure implementations determine how efficient a program is in terms of speed and memory usage. Pointer-based structures manage dynamic memory allocation flexibly, restricted linear structures enforce strict access rules, hierarchical trees optimize ordered searching, and hash functions enable constant-time data retrieval. Understanding these internal mechanics provides the foundation for the next chapter, which covers choosing and tailoring the right data structure to solve complex, real-world problems.

Tutorial Questions

1. Compare the operational time complexity and memory layouts of dynamic arrays and singly linked lists. Identify scenarios where each structure is preferred over the other.
2. Trace the following operations on an initially empty stack: `push(12)`, `push(7)`, `pop()`, `push(19)`, `push(3)`, `pop()`, `peek()`. Draw the stack layout at each step and state the value returned by the final `peek()` operation.
3. Explain why a standard array implementation of a queue can result in an $O(n)$

dequeue runtime performance. Show how a circular array structure resolves this issue using modular arithmetic.

4. Evaluate the following postfix expression step-by-step using a stack, showing the stack state after reading each token:

8 4 2 / * 3 + 5 −

5. Given an unweighted binary search tree containing keys $[50, 25, 75, 10, 30, 60, 80]$, construct the resulting tree topology. Demonstrate step-by-step node updates when deleting key 25.
6. Insert keys $[18, 26, 35, 9, 44, 27]$ into a hash table of size $M = 7$ using hash function $h(k) = k \pmod{7}$. Draw the resulting table structures under:
 - (a) Linear Probing Open Addressing.
 - (b) Separate Chaining.
7. A system developer uses a binary search tree to store symbol table keys. Under worst-case insertion conditions, operations degrade to $O(n)$ time. Explain why this degradation occurs and propose an architectural modification to ensure worst-case $O(\log n)$ performance.
8. High-performance caching layers use a Least Recently Used (LRU) policy, which requires fast element access and fast structural updates. Identify two data structures covered in this chapter that can be combined to support $O(1)$ search and $O(1)$ displacement operations, and explain how they interact.

Chapter 4

choose the appropriate data structure for modelling a given problem;

Selecting the appropriate data structure is a foundational decision in software engineering. Every computational problem exhibits distinct operational access patterns, memory constraints, and structural relationships among data elements. A data structure that excels at sequential data processing may perform poorly when required to perform frequent random lookups or arbitrary insertions. Consequently, software designers must analyze problem requirements systematically to select or compose an optimal data structure rather than relying on default choices.

This chapter establishes a practical methodology for modeling computational problems and mapping their operational requirements to concrete data structures. Building on the fundamental properties of built-in data structures, abstract data types, and concrete linear and non-linear implementations studied in previous chapters, this material synthesizes operational requirements with performance trade-offs. Students will learn to analyze workloads, data relationships, and physical resource constraints to select optimal representations.

Mastering this selection framework is critical for building scalable, high-performance software systems. Inappropriate choices lead to degraded computational efficiency, excessive memory consumption, and overly complex code logic. The principles established in this chapter prepare students for formal algorithm efficiency analysis in subsequent topics, bridging conceptual software design and rigorous performance evaluation.

4.1 Dimensions of Problem Requirements Analysis

When encountering a software modeling task, the software designer must decompose the problem requirements along four primary dimensions: computational operations, structural relationships, ordering constraints, and hardware resource limits.

4.1.1 Workload and Operational Patterns

Computational workloads differ based on the frequency and performance sensitivity of core operations:

- **Access and Search:** Does the problem require looking up elements by integer index, searching for specific data values, or retrieving records via unique alphanumeric keys?
- **Insertion and Deletion:** Are elements added or removed primarily at fixed boundaries (such as the start or end of a sequence) or at arbitrary interior locations?
- **Read-to-Write Ratio:** Is the workload read-heavy (dominated by search and traversal operations) or write-heavy (dominated by frequent state updates and structural modifications)?

4.1.2 Data Relationships and Structural Topology

Data elements possess inherent structural relationships that dictate the logical container topology:

- **Linear Relationships:** Elements exist in a strict sequential order, where each element except boundary nodes has a single predecessor and successor.
- **Hierarchical Relationships:** Elements exhibit parent-child dependencies, forming tree structures where a single root node branches into subtrees.
- **Associative Relationships:** Data is defined by key-value bindings, where lookup operations rely on abstract keys rather than physical positions.
- **Set Relationships:** Elements represent unordered collections of unique items, where set membership verification and mathematical set operations dominate.

4.1.3 Ordering and Uniqueness Constraints

Constraints on data ordering and duplication narrow the set of viable candidate data structures:

- **Insertion Order Preservation:** The collection must retain the exact sequence in which items were inserted into the system.
- **Sorted Order Maintenance:** Elements must remain continuously sorted according to a defined total ordering relation.
- **Uniqueness Constraints:** Duplicate values are strictly prohibited, requiring efficient membership testing during insertion.

4.1.4 Resource Constraints and Physical Hardware Characteristics

Resource limits frequently override theoretical operational efficiency considerations:

- **Memory Footprint and Pointer Overhead:** Node-based structures like a singly linked list or a binary search tree store memory references alongside payload data. In

contrast, dynamic arrays store elements contiguously without pointer overhead.

- **Cache Locality:** Modern CPUs fetch memory into hardware caches in contiguous blocks. The spatial proximity of data elements in memory, known as **cache locality**, enables arrays to execute significantly faster than pointer-based structures whose nodes are scattered across heap memory (Figure 4.1).
- **Allocation Predictability:** Dynamic reallocation strategies incur latency spikes when resizing. Fixed-size applications favor pre-allocated contiguous memory blocks.

Contiguous Memory Layout (Dynamic Array):

```
+-----+-----+-----+-----+-----+
| Data[0] | Data[1] | Data[2] | Data[3] | Data[4] | <-- Loaded in single
+-----+-----+-----+-----+-----+          cache line
Addr: 0x1000  0x1004    0x1008    0x100C    0x1010
```

Pointer-Based Node Layout (Linked List):

```
+-----+-----+          +-----+-----+          +-----+-----+
| Data[0] | Next |----> | Data[1] | Next |----> | Data[2] | Null |
+-----+-----+          +-----+-----+          +-----+-----+
Addr: 0x1000                Addr: 0x4800                Addr: 0x2200
                        (Heap Allocation Scatter -> Cache Misses)
```

Figure 4.1: Comparison of Contiguous Array Memory Layout and Pointer-Based Node Layout.

The initial mapping from problem characteristics to candidate data structures is summarized in Figure 4.2.

4.2 Comparative Analysis of Core Data Structures

Selecting an optimal data structure requires balancing execution time against spatial overhead. This fundamental compromise is known as the **time-space trade-off**, where an algorithm or data structure achieves faster operation execution by consuming additional memory, or conversely minimizes memory consumption at the expense of runtime performance.

For example, a hash table provides near-instantaneous search by reserving extra unallocated space in an underlying bucket array. Conversely, an unsorted dynamic array consumes minimal memory overhead but requires scanning every element sequentially to perform a search.

Table 4.1 provides a comparative reference of operational characteristics and memory profiles across standard linear and non-linear data structures.

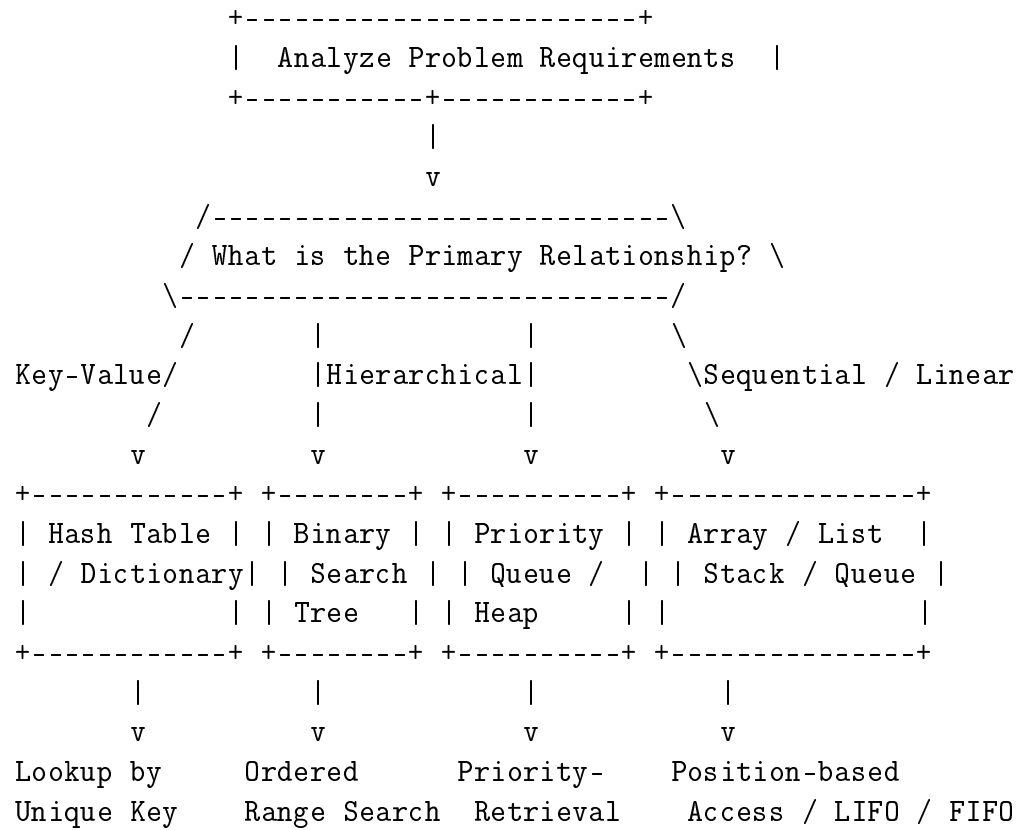


Figure 4.2: Taxonomy of Problem Requirements and Candidate Data Structures.

Table 4.1: Operational Complexity and Resource Profiles of Core Data Structures

Data Structure	Search / Lookup	Insertion	Deletion	Memory Overhead & Locality
Array	$O(1)$ by index; $O(n)$ by value	$O(n)$ arbitrary position	$O(n)$ arbitrary position	Zero pointer overhead; high cache locality.
Dynamic Array	$O(1)$ by index; $O(n)$ by value	$O(1)$ amortized at end; $O(n)$ arbitrary	$O(1)$ at end; $O(n)$ arbitrary	Low memory overhead; high cache locality.
Singly Linked List	$O(n)$ by index or value	$O(1)$ at head; $O(n)$ arbitrary	$O(1)$ at head; $O(n)$ arbitrary	One pointer reference per node; poor cache locality.
Doubly Linked List	$O(n)$ by index or value	$O(1)$ at ends; $O(1)$ given node reference	$O(1)$ at ends; $O(1)$ given node reference	Two pointer references per node; poor cache locality.
Stack	$O(n)$ general search	$O(1)$ push operation	$O(1)$ pop operation	Dependent on backing dynamic array or list container.
Queue	$O(n)$ general search	$O(1)$ enqueue operation	$O(1)$ dequeue operation	Dependent on backing ring buffer or list container.
Hash Table / Dictionary	$O(1)$ average time by key	$O(1)$ average time	$O(1)$ average time	Hash bucket capacity overhead; poor sequential locality.
Binary Search Tree	$O(\log n)$ average search	$O(\log n)$ average insertion	$O(\log n)$ average deletion	Two pointer references per node; supports sorted order.

4.3 Systematic Selection Methodology

To evaluate software requirements methodically, software developers follow a structured four-stage selection framework, illustrated in Figure 4.3.

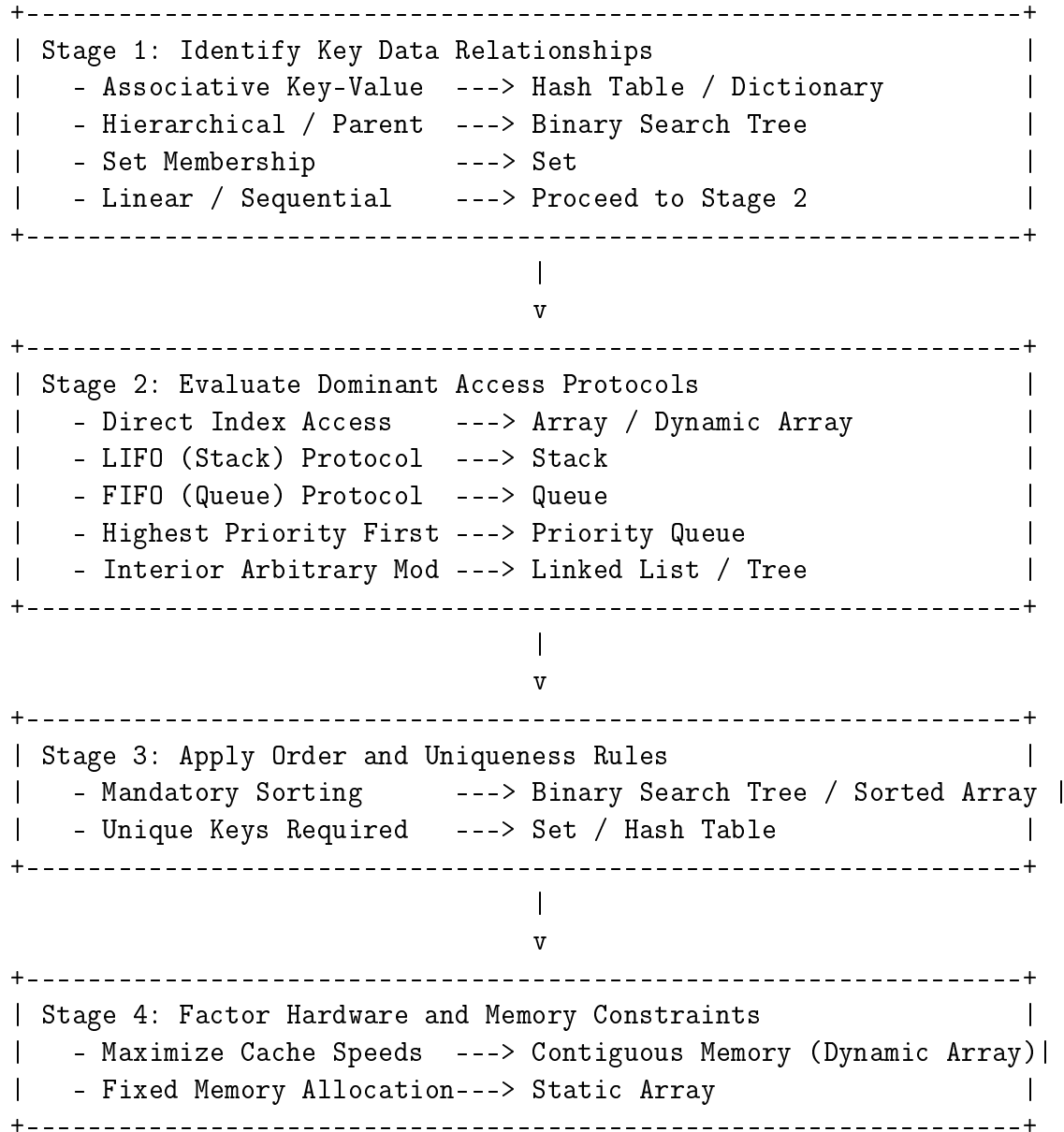


Figure 4.3: Four-Stage Decision Pipeline for Data Structure Selection.

The evaluation steps are executed in order:

1. **Identify Data Relationships:** Determine if data elements are bound by keys, hierarchical structures, set definitions, or simple sequences.
2. **Evaluate Operational Workloads:** Identify the most frequent operations (such as insertions, deletions, or searches) and choose structures optimized for those primary

access protocols.

3. **Apply Constraints:** Determine whether elements must remain strictly ordered or unique across the collection lifecycle.
4. **Assess Memory Overhead and Hardware Targets:** Factor in pointer allocation costs, cache locality requirements, and device memory limitations.

4.4 Worked Problem Modelling Scenarios

The following real-world software engineering scenarios demonstrate how to apply the systematic selection methodology to choose optimal data structures.

4.4.1 Scenario 1: Undo and Redo Subsystem for a Text Editor

Problem Specification: A text editor application requires an internal system to track user modifications, enabling unlimited undo and redo operations.

Requirement Analysis:

- Modifying text pushes a new state snapshot to the undo history.
- Performing an undo operation reverts the most recent modification, pushing the reverted state to a redo history.
- Access follows a strict Last-In, First-Out (LIFO) protocol. Access to intermediate historical states is not required.

Candidate Evaluation:

- **Dynamic Array / Stack:** Provides $O(1)$ push and pop operations at the boundary. Dynamic arrays store snapshots contiguously, maximizing memory access speed and eliminating pointer overhead.
- **Doubly Linked List:** Supports $O(1)$ boundary insertions and removals, but incurs pointer allocation overhead for every recorded edit operation.

Selection and Justification: A **stack** abstract data type implemented via a dynamic array is optimal. It offers $O(1)$ operational execution speed for LIFO access while minimizing memory overhead and leveraging processor cache locality.

4.4.2 Scenario 2: Real-time Operating System Task Scheduler

Problem Specification: An operating system scheduler manages active execution threads. Processes arrive dynamically with associated priority values. The system must continuously identify and extract the waiting process with the highest priority score.

Requirement Analysis:

- Dynamic task insertions occur continuously.
- Retrieval must always return the item with the highest priority score, rather than the oldest arrival.

- Linear searches across unsorted collections are unacceptable due to strict latency bounds.

Candidate Evaluation:

- **Unsorted Dynamic Array:** Insertions execute in $O(1)$ time, but locating the highest priority process requires scanning every element ($O(n)$ time complexity).
- **Sorted Array:** Locating and removing the highest priority item takes $O(1)$ time at the array boundary, but inserting new tasks requires shifting elements ($O(n)$ time complexity).
- **Priority Queue:** A **priority queue** abstract data type implemented via a heap or binary tree structure provides $O(\log n)$ insertion time and $O(\log n)$ priority extraction time.

Selection and Justification: A priority queue is chosen because it balances dynamic insertions and removals in logarithmic time, ensuring predictable performance under high workload concurrency.

4.4.3 Scenario 3: Academic Record Database with Range Queries

Problem Specification: A university administration portal stores student academic records identified by a unique Matriculation Number. The system requires constant-time lookups by student ID and must frequently produce ordered reports for ranges of Matriculation Numbers (for example, generating a roster for student IDs between 1000 and 2000).

Requirement Analysis:

- Exact key lookups must execute rapidly.
- Range queries require traversing data elements in sorted order within specified lower and upper bounds.

Candidate Evaluation:

- **Hash Table / Dictionary:** Delivers $O(1)$ average search time for individual key lookups. However, hash functions scatter keys unpredictably, making range queries inefficient ($O(n)$ full table scan).
- **Binary Search Tree:** Provides $O(\log n)$ search performance and maintains elements in sorted order, enabling efficient range traversals in $O(\log n + k)$ time, where k is the number of elements in the target range.
- **Hybrid Dual-Structure (Hash Table + Binary Search Tree):** Combines a hash table for $O(1)$ point lookups with a binary search tree for range queries, using duplicate references pointing to shared record memory.

Selection and Justification: If range queries occur frequently alongside single-record lookups, a **binary search tree** provides an excellent single-container solution. If single-item lookups heavily dominate the workload, a hybrid container system combining a hash table and a binary search tree offers the best overall performance, trading extra reference memory to achieve optimal lookup speeds.

4.5 Conclusion

Selecting the correct data structure requires a structured evaluation of operational access patterns, data relationships, ordering rules, and hardware limits. As demonstrated by the modeling scenarios, no single data structure performs optimally across all computational contexts. Software engineering requires evaluating operational trade-offs to select or combine data structures effectively.

While this chapter established qualitative selection criteria, evaluating algorithm scalability quantitatively requires formal mathematical models. The next chapter introduces **big-O notation** and formal algorithm analysis, providing the mathematical tools necessary to calculate upper bounds on execution time and memory usage across data structure operations.

Tutorial Questions

1. Compare dynamic arrays and doubly linked lists for a software module requiring frequent insertions in the middle of a collection versus one requiring end-only additions. Identify which structure is preferred in each case and explain your reasoning.
2. A developer chooses a hash table to implement a gaming leaderboard that must print the top ten highest scores in sorted order every second. Identify the primary structural limitation of hash tables in this scenario, and propose an alternative data structure that solves the problem efficiently.
3. Define the concept of cache locality and explain how memory layout influences the practical performance of contiguous arrays compared to pointer-based linked lists on modern processors.
4. A hospital triage system manages arriving emergency room patients based on assigned severity scores. Patients with higher severity scores must be treated before patients with lower scores, regardless of arrival time. Identify the most appropriate abstract data type for this system and justify your choice over a standard FIFO queue.
5. Consider a system workload consisting of 99% search operations and 1% insertion operations on a static dataset. Evaluate whether a sorted array or a binary search tree is more appropriate, factoring in execution time and memory footprint.
6. A system requires storing string key-value pairs on an embedded device with severely limited memory. Explain the time-space trade-off involved when deciding between a hash table and a sorted array of key-value tuples.
7. Design a hybrid data structure for an online shopping cart application that must support $O(1)$ item lookup by Product ID, preserve the chronological insertion sequence of added items, and allow fast deletion of items from any position in the cart. Explain how the individual data structures interact within your design.

Chapter 5

analyse simple algorithms and determine their efficiency using big-O notation; and

Selecting an appropriate data structure requires a mathematical framework to evaluate algorithmic efficiency independently of physical execution hardware. Measuring wall-clock runtime on a specific computer introduces confounding factors such as CPU clock frequency, compiler flags, operating system scheduling, and hardware cache structures. Algorithm analysis circumvents these empirical variables by expressing execution time and memory consumption as mathematical functions of the input size n .

This chapter establishes the principles of **asymptotic analysis**, focusing on mathematical notations used to bound performance as problem size grows toward infinity. It introduces formal definitions for **big-O notation**, Big- Ω notation, and Big- Θ notation. The chapter examines best-case, worst-case, and average-case analysis scenarios, and details formal step-counting techniques for analyzing iterative loops and simple recursive operations.

Mastering efficiency analysis enables computer scientists to predict how software implementations scale long before deployment. This mathematical grounding directly links the linear lists, trees, and hash tables examined in prior chapters to real-world application domains, laying the groundwork for analyzing complex algorithms such as data compression protocols in subsequent topics.

5.1 Principles of Algorithm Efficiency

Evaluating an algorithm requires measuring how its resource requirements scale as the size of the input n increases. The primary resources under evaluation are execution time (time complexity) and memory consumption (space complexity). Rather than measuring execution time in seconds, algorithm analysis counts primitive operations. A primitive operation is a low-level computation whose execution time is bounded by a constant, regardless of input size.

Examples of primitive operations include:

- Assigning a value to a variable.
- Performing a basic arithmetic operation (addition, subtraction, multiplication).

- Comparing two numerical values.
- Indexing into an array or dereferencing a memory reference.
- Returning a value from a function.

Let $T(n)$ represent the exact total count of primitive operations performed by an algorithm for an input of size n . For large values of n , low-order terms and constant coefficients exert negligible influence on the overall growth rate of $T(n)$. Asymptotic analysis focuses on the dominant term of $T(n)$ as n approaches infinity, isolating the underlying efficiency class of the algorithm.

5.2 Formal Asymptotic Notations

Asymptotic notation provides a mathematical language for bounding functions. Three primary notations are used to classify algorithm resource consumption: Big-O, Big-Ω, and Big-Θ.

5.2.1 Big-O Notation (Upper Bound)

Big-O notation defines an asymptotic upper bound on a function. It characterizes the mathematical ceiling of an algorithm's growth rate, ensuring that resource usage will not exceed a specific growth rate for sufficiently large inputs.

Formally, given two non-negative functions $f(n)$ and $g(n)$, $f(n) \in O(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

To establish that $f(n) = 3n^2 + 5n + 8$ is $O(n^2)$, appropriate constants c and n_0 must be identified. Observing that for all $n \geq 1$:

$$5n \leq 5n^2 \quad \text{and} \quad 8 \leq 8n^2$$

Adding these inequalities yields:

$$3n^2 + 5n + 8 \leq 3n^2 + 5n^2 + 8n^2 = 16n^2$$

Setting $c = 16$ and $n_0 = 1$ demonstrates that $f(n) \leq 16n^2$ for all $n \geq 1$. Thus, $f(n) \in O(n^2)$.

5.2.2 Big-Ω Notation (Lower Bound)

Big-Ω notation defines an asymptotic lower bound on a function. It describes the minimum growth rate an algorithm guarantees for large input sizes.

Formally, $f(n) \in \Omega(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$f(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0$$

For the function $f(n) = 3n^2 + 5n + 8$, observing that $5n + 8 \geq 0$ for $n \geq 1$ gives:

$$3n^2 + 5n + 8 \geq 3n^2$$

Setting $c = 3$ and $n_0 = 1$ confirms that $f(n) \in \Omega(n^2)$.

5.2.3 Big- Θ Notation (Tight Bound)

Big- Θ notation defines an asymptotically tight bound. A function $f(n)$ belongs to $\Theta(g(n))$ if and only if $f(n)$ is bounded both above and below by scaled multiples of $g(n)$.

Formally, $f(n) \in \Theta(g(n))$ if there exist positive constants $c_1 > 0$, $c_2 > 0$, and $n_0 \geq 1$ such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0$$

Because $f(n) = 3n^2 + 5n + 8$ is both $O(n^2)$ with $c_2 = 16$ and $\Omega(n^2)$ with $c_1 = 3$, it follows directly that $f(n) \in \Theta(n^2)$.

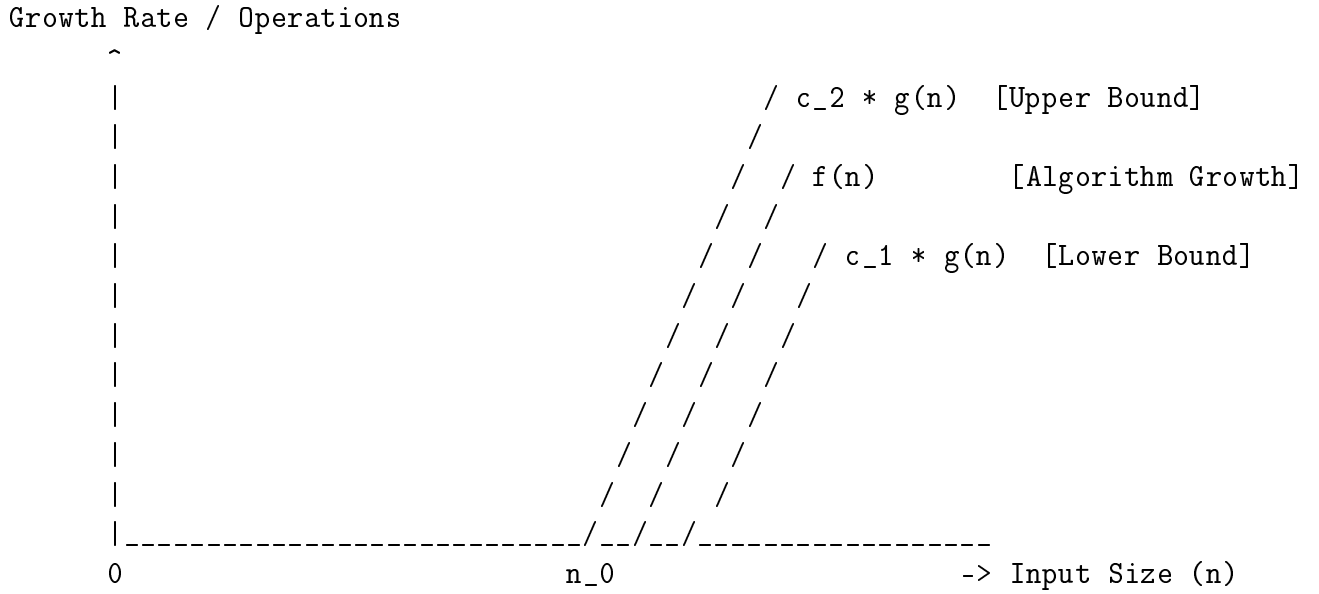


Table 5.1: Comparison of Formal Asymptotic Notations

Notation	Mathematical Inequality	Bound Type	Practical Interpretation
$O(g(n))$	$f(n) \leq c \cdot g(n)$ for $n \geq n_0$	Upper bound	Worst-case growth rate ceiling. Resource usage will not exceed this rate.
$\Omega(g(n))$	$f(n) \geq c \cdot g(n)$ for $n \geq n_0$	Lower bound	Best-case growth rate floor. Resource usage will not drop below this rate.
$\Theta(g(n))$	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for $n \geq n_0$	Tight bound	Exact growth rate matching $g(n)$ within constant factor bounds.

5.3 Analysis Scenarios: Worst-Case, Best-Case, and Average-Case

An algorithm's runtime can vary significantly based on the specific arrangement of input elements, even when input size n remains constant. Three performance scenarios are evaluated:

- **worst-case time complexity:** The maximum number of primitive operations performed over all valid inputs of size n . It provides a strict upper boundary guarantee for system safety and mission-critical design.
- **best-case time complexity:** The minimum number of primitive operations performed over all valid inputs of size n . It represents ideal conditions but rarely serves as a useful benchmark for general software performance.
- **average-case time complexity:** The expected number of primitive operations executed over all inputs of size n , weighted by their probability distribution.

Consider a linear search operation executed on an unsorted array containing n elements. The algorithm inspects each element sequentially from index 0 to index $n - 1$ until the target element is found or the array ends.

- **Best-Case:** The target element resides at index 0. The algorithm completes after 1 comparison, yielding $O(1)$ time complexity.
- **Worst-Case:** The target element is located at index $n - 1$ or is entirely absent from the array. The algorithm performs n comparisons, yielding $O(n)$ time complexity.
- **Average-Case:** Assuming the target is equally likely to be at any index from 0 to $n - 1$, the expected number of comparisons is calculated as:

$$\text{Average Comparisons} = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \left(\frac{n(n+1)}{2} \right) = \frac{n+1}{2}$$

Dropping constant factors and low-order terms establishes an average-case time complexity of $O(n)$.

5.4 Common Complexity Classes and Growth Rates

Algorithms are categorized into standard complexity classes based on their dominant growth terms. Ordering these classes from most efficient to least efficient yields:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

The relative growth rates of these complexity functions are illustrated below:

Resource Usage $T(n)$

^
|

/ $O(2^n)$ Exponential

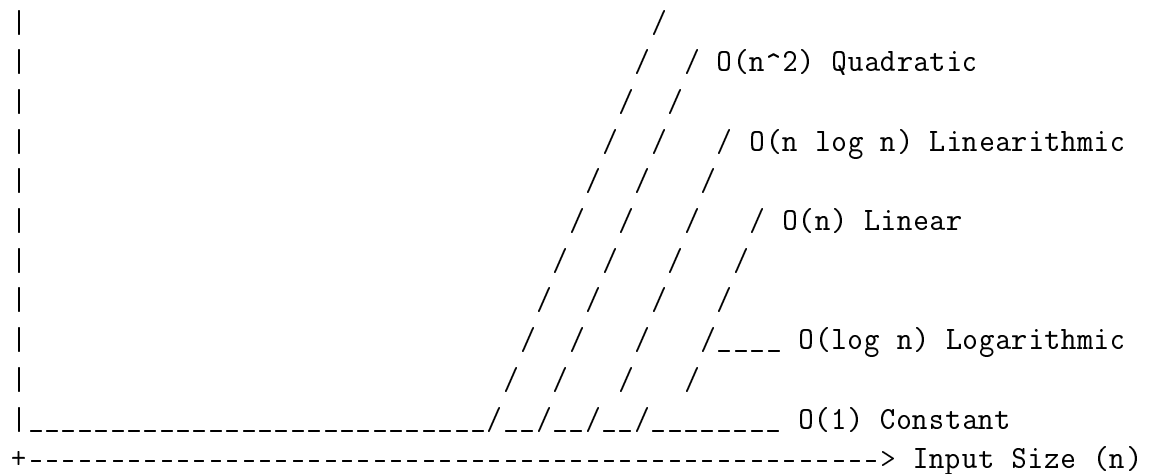


Table 5.2: Comparison of Growth Classes Across Operations Count

Complexity Class	$n = 10$	$n = 100$	$n = 1,000$	Common Data Structure Context
$O(1)$ Constant	1	1	1	array index lookup, stack push/pop
$O(\log n)$ Logarithmic	≈ 3.3	≈ 6.6	≈ 10	binary search tree lookup, binary search
$O(n)$ Linear	10	100	1,000	singly linked list traversal, linear search
$O(n \log n)$ Linearithmic	≈ 33	≈ 664	$\approx 9,966$	Merge sort, heap operations
$O(n^2)$ Quadratic	100	10,000	1,000,000	Insertion sort, nested matrix operations
$O(2^n)$ Exponential	1,024	1.27×10^{30}	1.07×10^{301}	Exhaustive recursive search

5.5 Analyzing Iterative Algorithms

Determining the time complexity of iterative algorithms requires counting operations inside loop structures and expressing total operations as algebraic sums.

5.5.1 Sequential Loops

When code blocks execute sequentially, their individual time complexities are summed. The dominant term dictates the overall upper bound:

$$T(n) = T_1(n) + T_2(n) \implies O(\max(f_1(n), f_2(n)))$$

5.5.2 Nested Loops with Independent Bounds

When loops are nested and the inner loop bound does not depend on the outer loop counter, total operations equal the product of the iteration counts.

```
def matrix_fill(matrix, n):
    for i in range(n):          # Outer loop executes n times
        for j in range(n):      # Inner loop executes n times
            matrix[i][j] = 0    # O(1) primitive assignment
```

The inner assignment runs n times for each outer loop iteration. Total primitive operations equal $n \times n = n^2$. The worst-case time complexity is $O(n^2)$.

5.5.3 Dependent Nested Loops

When loop counters interact, the inner loop bounds depend on the outer loop variable. Total execution count is determined by formulating and evaluating a summation.

Consider the following triangular loop construct:

```
def print_pairs(arr, n):
    for i in range(n):          # Outer loop runs for i = 0 to n-1
        for j in range(i + 1, n): # Inner loop runs from i + 1 to n-1
            print(arr[i], arr[j]) # Primitive O(1) operation
```

Analyzing iteration counts per outer loop step:

- When $i = 0$, inner loop executes $(n - 1)$ times.
- When $i = 1$, inner loop executes $(n - 2)$ times.
- When $i = n - 2$, inner loop executes 1 time.
- When $i = n - 1$, inner loop executes 0 times.

Total primitive operations are calculated by summing the arithmetic series:

$$T(n) = \sum_{i=0}^{n-1} (n - 1 - i) = (n - 1) + (n - 2) + \cdots + 1 + 0 = \frac{n(n - 1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

Discarding low-order terms and constant factors reveals a time complexity of $O(n^2)$.

5.5.4 Logarithmic Loops

Loops that scale their counter variable via multiplication or division in each step yield logarithmic time complexities.

```
def divide_step(n):
    count = 0
    i = n
    while i > 1:
```

```

count += 1
i = i // 2 # Integer division by 2
return count

```

Let k be the number of iterations executed before $i \leq 1$. The variable i takes the values $n, n/2, n/4, \dots, n/2^k$. The loop terminates when:

$$\frac{n}{2^k} \leq 1 \implies 2^k \geq n \implies k = \lceil \log_2 n \rceil$$

Because k total steps are executed, the overall time complexity is $O(\log n)$.

5.6 Analyzing Recursive Algorithms

Recursive algorithms call themselves on smaller subproblems. Analyzing their time complexity requires defining a **recurrence relation**, which expresses the overall execution time $T(n)$ in terms of $T(k)$ for smaller input sizes $k < n$.

5.6.1 Expansion Method for Recurrences

The expansion method involves repeatedly substituting a recurrence equation into itself until a recognizable pattern emerges, terminated by a base case.

Consider recursive binary search operating on a sorted array of size n . The algorithm evaluates the middle element and recursively searches either the left or right half:

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ T(n/2) + c & \text{if } n > 1 \end{cases} \quad (5.1)$$

where c represents the constant time taken for middle element comparisons, and d represents base case execution.

Expanding $T(n)$ step by step:

$$T(n) = T(n/2) + c$$

Substituting $T(n/2) = T(n/4) + c$:

$$T(n) = (T(n/4) + c) + c = T(n/4) + 2c$$

Substituting $T(n/4) = T(n/8) + c$:

$$T(n) = (T(n/8) + c) + 2c = T(n/8) + 3c$$

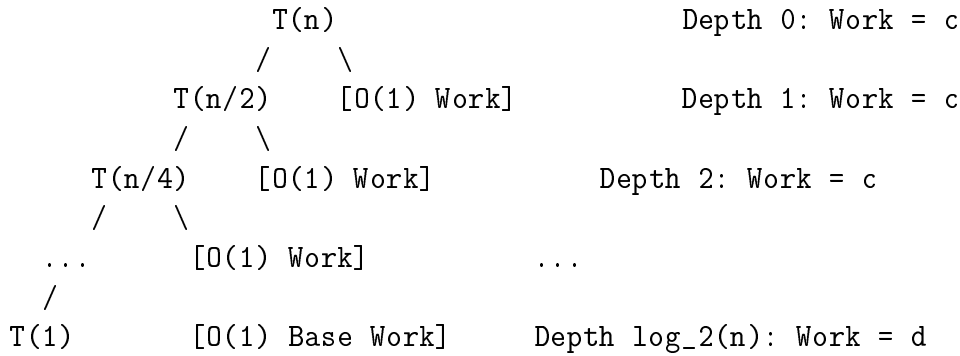
After k expansions, the general relation is:

$$T(n) = T(n/2^k) + k \cdot c$$

The expansion reaches the base case when $n/2^k = 1$, which occurs when $2^k = n$, or $k = \log_2 n$. Substituting $k = \log_2 n$ back into the expanded equation yields:

$$T(n) = T(1) + c \log_2 n = d + c \log_2 n$$

Removing constants and non-dominant terms yields a worst-case time complexity of $O(\log n)$.



Total Time Complexity: sum of work across all depths = $O(\log n)$

5.7 Space Complexity and Auxiliary Memory

Space complexity analyzes the total memory allocated by an algorithm as a function of input size n . Total space complexity comprises two distinct components:

- **Input space:** Memory occupied by the original input data structure.
- **auxiliary space:** Temporary or additional memory allocated during execution for algorithm processing, including working dynamic arrays, stack memory, or extra references.

Algorithm design prioritizes minimizing auxiliary space. For instance, an in-place sorting algorithm requires $O(1)$ auxiliary space beyond the input array, whereas non-in-place algorithms require $O(n)$ auxiliary space to allocate secondary arrays.

5.7.1 Call Stack Space in Recursive Algorithms

Recursive execution consumes auxiliary memory on the system call stack. Each active function call allocates a stack frame storing local variables, parameters, and return addresses. The auxiliary space complexity of a recursive algorithm equals the maximum depth of its call stack multiplied by the memory consumed per frame.

For instance, computing a factorial iteratively requires $O(1)$ auxiliary space:

```
def iterative_factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

In contrast, the recursive formulation allocates n stack frames, generating $O(n)$ auxiliary space complexity:

```
def recursive_factorial(n):
    if n <= 1:
        return 1
    return n * recursive_factorial(n - 1)  # Max stack depth = n
```

Iterative Call Stack ($O(1)$ Auxiliary Space):

```
+-----+
| iterative_factorial(n): frame reused each step |
+-----+
```

Recursive Call Stack ($O(n)$ Auxiliary Space):

```
+-----+
| recursive_factorial(1)  [Depth n: Base Case]  |
+-----+
| ...                               |
+-----+
| recursive_factorial(n-1) [Depth 2]            |
+-----+
| recursive_factorial(n)  [Depth 1: Initial Call] |
+-----+
```

Table 5.3: Space Complexity Trade-offs Across Common Data Structure Operations

Data Structure / Operation	Time Complexity	Auxiliary Space	Primary Source of Space Usage
Array Indexing	$O(1)$	$O(1)$	Direct offset calculation
Linked List Traversal	$O(n)$	$O(1)$	Single pointer reference tracking
Recursion on Balanced Tree	$O(\log n)$	$O(\log n)$	Function call stack depth
Recursion on Unbalanced Tree	$O(n)$	$O(n)$	Degenerate call stack depth
hash table Lookup	$O(1)$ average	$O(n)$ total	Hash buckets and entry arrays

Evaluating both time complexity and space complexity reveals essential **time-space trade-off** principles. Choosing an optimal algorithm involves balancing the computational speed of time execution against the physical memory limits imposed by auxiliary space allocation.

5.8 Conclusion

Asymptotic analysis provides the mathematical framework for measuring algorithm performance across scaling input sizes. By classifying operations into time and space complexity bounds such as Big-O, Big-Ω, and Big-Θ, developers can rigorously evaluate algorithmic performance independently of underlying physical hardware. Understanding these analysis methods enables precise comparative evaluations across fundamental data structures.

This efficiency analysis framework serves as a core decision tool when selecting and constructing data structures for specialized software applications. The mathematical tools established in this chapter will be applied directly to evaluate domain-specific operations in the next chapter, which explores application domains such as data compression, graph indexing, and dynamic memory management.

Tutorial Questions

1. Explain why measuring the wall-clock execution time of an algorithm on a physical machine is insufficient for determining algorithm efficiency. Identify three machine-dependent factors that distort empirical measurements.
2. Formal mathematical definitions specify upper, lower, and tight performance bounds.
 - (a) State the formal mathematical definition of Big-O notation.
 - (b) Using the formal definition, prove that $f(n) = 7n^2 + 11n + 5$ is $O(n^2)$ by selecting valid positive constants c and n_0 .
3. Compare worst-case, best-case, and average-case time complexities using linear search on an array of size n as an example. Explain why worst-case time complexity is preferred for real-time safety systems.
4. Analyze the worst-case time complexity of the following code fragment by performing exact operation step counting:

```
def execute_loops(n):  
    total = 0  
    for i in range(n):  
        j = 1  
        while j < n:  
            total += arr[i] + j  
            j = j * 2  
    return total
```

5. A recursive algorithm produces the recurrence relation $T(n) = T(n - 1) + c$ for $n > 1$, with base case $T(1) = d$.
 - (a) Use the expansion method to solve for the closed-form complexity of $T(n)$.
 - (b) Identify an operation on a linear data structure that exhibits this recurrence pattern.

6. Explain the distinction between total space complexity and auxiliary space complexity. Describe a scenario where an iterative algorithm and a recursive algorithm perform identical computations with the same time complexity but different auxiliary space complexities.
7. A developer must choose between two algorithms for processing a dataset of n elements: Algorithm A has a time complexity of $O(n^2)$ with $O(1)$ auxiliary space, while Algorithm B has a time complexity of $O(n \log n)$ with $O(n)$ auxiliary space. Explain how dataset scale and hardware memory constraints should guide the selection of the appropriate algorithm.

Chapter 6

apply the knowledge of data structures to other application domains like data compression

Data structures serve as foundational building blocks across computer systems. While basic implementations demonstrate individual operations such as insertion, deletion, and traversal, practical software applications demand composite configurations. Real-world engineering domains, including data compression, high-performance caching, database search engines, and network routing, rely on combining elementary structures into specialized software architectures.

Data compression algorithms reduce storage overhead and network bandwidth requirements by reorganizing data using specialized tree structures and priority queues. Memory management subsystems maintain real-time execution speeds through composite cache structures that pair key lookup tables with ordering lists. Search engines and network routers utilize specialized search trees and graph representations to evaluate millions of queries and routing decisions per second.

This chapter applies foundational data structures to solve algorithmic problems across system domains. By examining these implementations in context, this chapter demonstrates the practical application of asymptotic optimization, structural trade-offs, and object design principles introduced throughout this course.

6.1 Data Compression via Optimal Prefix Trees

Data compression is the process of encoding information using fewer bits than the original representation. Lossless data compression guarantees that the original data can be perfectly reconstructed from the compressed representation. A fundamental approach to lossless compression relies on variable-length character encoding, where frequently occurring characters are assigned shorter bit sequences, while rarer characters receive longer bit sequences.

To ensure unambiguous decoding, variable-length codes must satisfy the prefix property.

A **prefix code** is a coding system in which no valid code word is a prefix of any other valid code word. This property allows a continuous stream of compressed bits to be parsed sequentially without delimiter tokens.

6.1.1 Huffman Coding Algorithm

Huffman coding is an algorithmic method for constructing optimal prefix codes. The algorithm uses a min-heap priority queue to build a full binary tree bottom-up, based on the relative frequencies of input symbols.

The tree construction procedure follows a deterministic sequence:

1. Compute the occurrence frequency of each unique symbol in the input source.
2. Create a leaf node for each symbol and insert all leaf nodes into a min-heap priority queue ordered by frequency.
3. While the priority queue contains more than one node:
 - (a) Extract the two nodes with the lowest frequencies (q_1 and q_2).
 - (b) Create a new internal node with a frequency equal to the sum of q_1 and q_2 .
 - (c) Set q_1 as the left child and q_2 as the right child of the internal node.
 - (d) Insert the new internal node back into the min-heap.
4. The remaining single node in the priority queue represents the root of the complete Huffman tree.

Traversing the resulting binary tree assigns binary digits to edges: left branches represent bit 0 and right branches represent bit 1. The binary string accumulated from the root to a leaf node forms the unique variable-length prefix code for that symbol.

6.1.2 Worked Example: Encoding "ABRACADABRA"

Consider compressing the 11-character string "ABRACADABRA". Standard fixed-length ASCII encoding allocates 8 bits per character, requiring $11 \times 8 = 88$ bits.

First, tally character frequencies:

Table 6.1: Symbol Frequency Table for "ABRACADABRA"

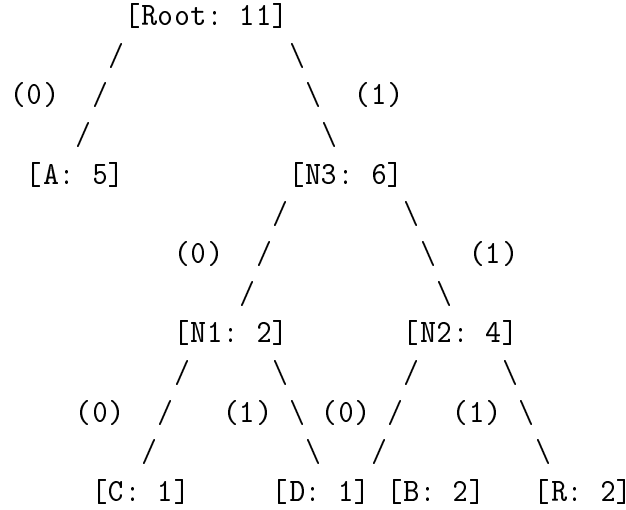
Symbol	Frequency
A	5
B	2
R	2
C	1
D	1

Next, construct the Huffman tree using a min-heap:

1. Initial min-heap contents (ordered by frequency): $[C(1), D(1), B(2), R(2), A(5)]$.

2. Extract $C(1)$ and $D(1)$. Create internal node N_1 with frequency $1 + 1 = 2$. Insert $N_1(2)$. Min-heap: $[B(2), R(2), N_1(2), A(5)]$.
3. Extract $B(2)$ and $R(2)$. Create internal node N_2 with frequency $2 + 2 = 4$. Insert $N_2(4)$. Min-heap: $[N_1(2), N_2(4), A(5)]$.
4. Extract $N_1(2)$ and $N_2(4)$. Create internal node N_3 with frequency $2 + 4 = 6$. Insert $N_3(6)$. Min-heap: $[A(5), N_3(6)]$.
5. Extract $A(5)$ and $N_3(6)$. Create root node R_{root} with frequency $5 + 6 = 11$.

The finalized Huffman binary tree layout is shown below:



Navigating from the root to each leaf yields the code table:

Symbol	Frequency	Huffman Code	Total Bits
A	5	0	5
B	2	110	6
R	2	111	6
C	1	100	3
D	1	101	3

The encoded string representation becomes:

0 (A) 110 (B) 111 (R) 0 (A) 100 (C) 0 (A) 101 (D) 0 (A) 110 (B) 111 (R) 0 (A)

The total storage consumed is $5 + 6 + 6 + 3 + 3 = 23$ bits. Compared to the uncompressed 88 bits, Huffman coding achieves a 73.86% space reduction for this message.

6.2 Composite Data Structures for High-Performance Caching

In-memory caching improves subsystem throughput by storing frequently queried data in fast RAM. Because cache memory capacity is limited, a cache must execute eviction

strategies when full. The **least recently used cache** (LRU cache) evicts the item that has not been accessed for the longest duration.

An LRU cache requires two operations to execute in $O(1)$ worst-case time complexity:

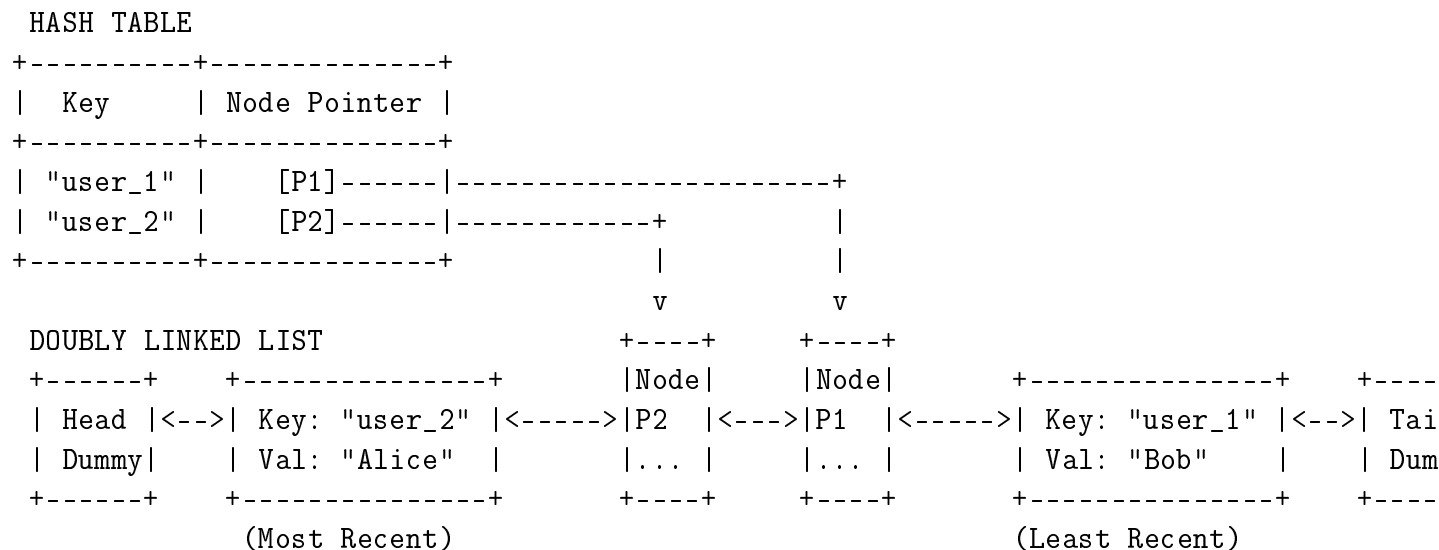
- `get(key)`: Retrieve the item value associated with the key and mark it as most recently used.
- `put(key, value)`: Insert or update a key-value pair. If capacity is exceeded, evict the least recently used item.

A single foundational data structure cannot satisfy both operational constraints simultaneously. Searching a doubly linked list requires $O(n)$ time, while a standard hash table cannot track item access recency in $O(1)$ time.

The optimal architectural solution couples a hash table with a doubly linked list.

6.2.1 LRU Cache Structural Integration

The doubly linked list maintains the items in sequence of access recency. The head of the list stores the most recently accessed item, while the tail stores the least recently accessed item. The hash table maps search keys directly to list node pointers, bypassing linear list traversal.



6.2.2 Operational Execution Trace

The composite LRU cache executes its core methods through joint state manipulation:

Get Operation

1. Query the hash table with the key. If the key is absent, return a cache miss status.
2. If the key is present, extract the direct memory pointer to the list node.
3. Detach the node from its current position in the doubly linked list by modifying its neighbors' pointers in $O(1)$ time.

4. Splice the node directly behind the list head node (marking it most recently used).
5. Return the stored value.

Put Operation

1. Query the hash table with the key. If the key exists, update its value field and promote the node to the list head.
2. If the key does not exist:
 - (a) If the cache count equals capacity, retrieve the node preceding the tail dummy node (the least recently used node).
 - (b) Delete the key of the tail node from the hash table.
 - (c) Remove the tail node from the doubly linked list in $O(1)$ time.
 - (d) Allocate a new node containing the key and value, splice it behind the head node, and add its pointer reference to the hash table.

6.3 Database Indexing and Search Engine Retrieval

Information retrieval and database systems process non-numeric data queries, such as text searches and prefix matching. Generic binary search trees or arrays are inefficient for large-scale string processing due to string comparison overheads. Specialized indexing structures address these operational constraints.

6.3.1 Inverted Indexing

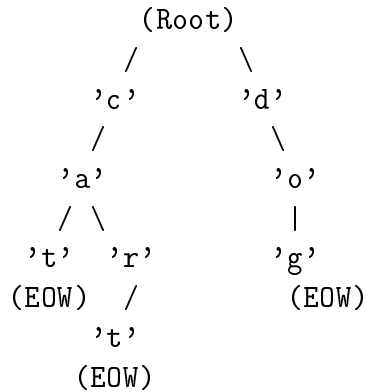
An **inverted index** is a dictionary structure optimized for text retrieval across large document collections. Instead of mapping documents to contained words, an inverted index maps distinct terms to a list of document identification tags where those terms appear. This structure is known as a postings list.

TERM DICTIONARY (Hash Table / Tree)	POSTINGS LISTS (Singly Linked Lists)
+-----+	+----+ +----+ +----+
"algorithm" ----->	12 ---> 45 ---> 89
+-----+	+----+ +----+ +----+
"binary" ----->	3 ---> 45
+-----+	+----+ +----+
"complexity" ----->	12 ---> 99
+-----+	+----+ +----+

Constructing an inverted index uses a hash table or search tree for the primary term dictionary, while storing postings as dynamic arrays or singly linked lists. To execute a boolean search query such as "algorithm AND binary", the engine retrieves the posting lists for both terms and performs a linear list intersection algorithm in time proportional to the length of the lists.

6.3.2 Prefix Search via Tries

A **trie** (also termed a prefix tree) is a tree-based search data structure where nodes represent character keys rather than entire data items. The root node represents an empty key, and child nodes represent single-character extensions.



In the trie above, the stored words are "cat", "cart", and "dog". Nodes labeled (EOW) represent the end of a valid word.

Tries enable optimal string searching and auto-complete functionality. Searching for a string of length k takes $O(k)$ time complexity, independent of the total number of keys stored in the dataset.

6.4 Data Structure Selection in Applied Domains

Selecting data structures requires evaluating operations, time complexities, and memory requirements against application objectives. Table 6.3 compares the domain-specific data structures discussed in this chapter.

6.5 Graph Structures in Network Routing

Graph structures model connected networks such as the internet, road systems, and social graphs. Modern routing protocol engines rely on efficient graph representations coupled with priority queues to evaluate optimal signal pathways.

Dijkstra's shortest path algorithm calculates the minimum path distance from a single source vertex to all other vertices in a weighted graph.

The graph representation dictates algorithmic efficiency:

- **Adjacency Matrix:** An $n \times n$ matrix providing $O(1)$ edge weight lookups, but requiring $O(V^2)$ total space complexity.
- **Adjacency List:** Array of linked lists storing only existing edges, consuming $O(V + E)$ space complexity, where V is vertices and E is edges.

SAMPLE GRAPH

ADJACENCY MATRIX

ADJACENCY LIST

Table 6.3: Comparison of Domain-Specific Data Structures

Structure	Core Operations	Time Complexity	Primary Domain
Huffman Tree	Bit insertion, Prefix code parsing	$O(n \log k)$ build, $O(k)$ decode per character	Lossless text and media compression
LRU Cache	Key lookups, Node promotion and eviction	$O(1)$ for get and put	In-memory performance caching
Inverted Index	Dictionary lookup, List intersection	$O(k)$ key search, $O(L_1 + L_2)$ intersection	Full-text search engine indexing
Trie	String search, Prefix matching	$O(k)$ key lookup ($k =$ string length)	Auto-complete, IP packet routing

(A) --- (B)

| /

| /

(C) --- (D)

A B C D

A 0 1 1 0

B 1 0 1 1

C 1 1 0 0

D 0 1 0 0

[A] -> B -> C

[B] -> A -> C -> D

[C] -> A -> B

[D] -> B

When Dijkstra's algorithm uses a flat array to scan unvisited vertices, the minimum distance selection takes $O(V)$ time per vertex, producing an overall runtime complexity of $O(V^2)$.

By pairing an adjacency list graph representation with a min-heap priority queue, finding the minimum distance vertex executes in $O(\log V)$ time. This optimization reduces the overall algorithm execution time complexity to $O((V + E) \log V)$, enabling real-time routing over massive networks.

Tutorial Questions

1. Explain the prefix property in variable-length code systems. Describe the functional problems that arise during decoding if a set of codes violates this property.
2. Construct a Huffman code tree for a document containing the following character counts: E: 12, T: 9, A: 8, I: 4, O: 2. List the binary code generated for each character and compute the total bits required to encode the source document.
3. An LRU cache with a total capacity of 3 items processes the following sequence of execution calls:


```
put("A", 1), put("B", 2), put("C", 3), get("A"), put("D", 4)
```

Draw the contents of both the hash table and the doubly linked list after each operation completes. Identify which key is evicted upon inserting ("D", 4).

4. Compare the storage efficiency and time performance of a hash table versus a trie for storing a dictionary of 500,000 text strings. Highlight conditions under which a trie is preferred.
5. Explain how an inverted index executes multi-word intersection queries (such as "data AND structure"). State the worst-case time complexity of intersecting two posting lists of lengths M and N .
6. Analyze the algorithmic impact of data structure selection in Dijkstra's shortest path algorithm. Contrast the theoretical performance of an implementation using a linear array versus one using a min-heap priority queue.
7. A network router needs to match incoming IP addresses to a table of 100,000 network prefixes in real time. Choose an appropriate data structure for this application domain, justify your selection over alternative structures, and analyze its search time complexity.

Conclusion

Data structures are non-isolated theoretical models; they serve as structural primitives that power software engineering solutions across modern applications. As demonstrated throughout this chapter, complex domain applications, including lossless compression engines, operating system memory caches, full-text search indexes, and network packet routers, require combining foundational data structures to meet real-world computational constraints. Combining hashing, tree models, linked sequences, and priority queues allows systems to optimize execution time and space utilization simultaneously, translating theoretical complexity bounds into scalable software architectures. Having examined how composite data structures optimize resource efficiency in production systems, the next step is to explore how algorithmic design paradigms—such as greedy strategies, divide-and-conquer, and dynamic programming—leverage these underlying structures to solve complex computational problems.